边听边说：全双工语音交互如何从轮次流水线长成连续多流智能体

语音助手已经存在很久了。你说话，它听完；它思考，你等待；它念完，你再开口。这条流水线在工程上极其成功，却在体验上始终不像真人。真人对话不是排队过闸机，而是两个人在同一条时间轴上同时呼吸：你还没说完，对方已经"嗯"了一声；你说到一半改主意，对方立刻让路；你停顿三秒想词，对方不会抢话；你一边听导航播报，一边改目的地，系统必须在出声的同时继续听。

所以全双工要解决的，从来不是再把半双工做快一点。它要改写的是时间本身：**对话从轮次排队，变成连续时间上的状态演化**。 音频生成、端到端建模、话轮控制、边想边说、工具调用、多模态感知，都是这条改写被迫长出来的模块——不是产品路线图上依次打勾的功能清单。

下面不按论文发表时间堆模型，而是先把容易混在一起的概念拆开，再看领域怎么一步步把假轮次拆掉。

一、坐标系：端到端与全双工是两回事

很多人把路线画成一条直线：级联 → 半级联 → 端到端 → 全双工。这条直线暗示端到端等于全双工。这是错的。

更准确的是一张二维表。横轴是**架构：信息要不要被迫经过文本**；纵轴是**交互：系统说话时还能不能继续听**。四个格子都真实存在。

架构 \ 交互	Half-Duplex（同一时刻只占听或说）	Full-Duplex（说话时仍持续收用户声）
Cascade	经典 ASR→LLM→TTS 助手	级联骨架 + 话轮控制器
半级联 / E2E S2S	流式语音大模型，但仍按轮切段	原生多流 / 状态内化双工

这个区分之所以要紧，是因为失败形态完全不同。端到端没做好，你会丢掉副语言：情绪、犹豫、笑声、口音、环境音。全双工没做好，你会听得懂却不会相处：该停不停、该接不接、该附和却抢答、该让路却硬说完。一个系统可以端到端极强却仍是半双工；也可以全双工体验很好但内容仍走级联。

再纠正一个更常见的混淆：**Streaming ≠ Full-Duplex**。把 ASR、LLM、TTS 全部改成流式，首包延迟会下降，但只要逻辑仍是用户说完再答，同一时刻系统仍只占一个方向，它就仍是半双工。真正的全双工，时间轴上必须允许重叠：

时间线	前段	中段	后段
用户	静音 / 听系统	开口（可与系统重叠）	继续或停
系统	先开口	仍在说，同时继续听	让路 / 继续 / 忽略

重叠段就是打断、附和、第三方拒识真正发生的地方。

全双工要同时回答四个问题，而不是只回答怎么生成一段语音：

问题	含义
What to hear	听到什么：词、情绪、犹豫、噪声、他人说话
When to speak	何时开口、何时停、何时只"嗯"一下
What to say	说什么：推理、记忆、工具结果
How to speak	怎么说：音色、韵律、笑声、犹豫

后面出现的模块，大体都能收进这四问。

二、领域真正卡住的不是模型名字，而是三组本质矛盾

2.1 传统管线的三宗罪

第一宗罪是**延迟**。级联把理解、决策、合成串成三段，交互间隙经常落到秒级。人对自然回应的体感大约在两百毫秒量级。差一个数量级，对话就会像呼叫中心，不像对面坐着一个人。

第二宗罪是**文本信息瓶颈**。ASR 的任务是产出干净可检索的文本。它天然倾向于抹掉对写字无用、对相处极有用的信息：升调还是降调、笑了一下还是冷笑、说到一半卡壳、背景是不是很吵。后面的 LLM 再聪明，也只能在被阉割后的文本上推理。

第三宗罪是**轮次假设失败**。系统把世界切成用户 turn / 助手 turn。真实对话却充满重叠、打断、附和、句中停顿。静音不等于话轮结束；短促的"嗯"不等于抢答；用户突然开口也不等于整段旧答案作废后的新一轮问答——更像是同一次相遇里的局部改道。

2.2 交互四轴：搭出来之后验什么

即使系统号称全双工，仍可能只会某一种双工。评测与产品真实体感，通常落在四根轴上：

交互轴	要求	一句话
Pause	句中停顿时别抢	忍得住
Turn-taking	真说完时快接	接得住
Backchannel	长叙述时短附和	嗯得对
Interruption	被打断时让路并接上	让得开

这四轴内部互相打架。Pause 要求你少开口，Turn-taking 要求你快开口；Backchannel 要求你开口但要短，Interruption 要求你立刻停自己的长输出。所以预训练似然最大化几乎不可能顺便把四轴一起学对——后面会看到，为什么需要显式状态、专用数据，甚至后训练强化学习。

还有一个测量上的陷阱：同一种接管率指标，在 Pause/Backchannel 上要低，在 Turn/Interruption 上要高。快不等于全面好。Moshi 一类多流模型往往接话和打断很快，却容易在停顿上抢话、在附和上变成抢答。保守的状态机可能 Pause 较好，却几乎不"嗯嗯"，接话又慢又漏。

2.3 三大底层矛盾决定了技术为什么会分叉

只问如何实现全双工太空。更底层的是三场必打的仗：

延迟 vs 智能。 想得越深，开口越晚。如果不把思考和发声拆开并行，系统要么像聪明人但反应慢，要么像反应快但脑子空。

听 vs 说。 半双工把二者串行；全双工要并行。并行立刻带出自听问题：扬声器里的自己会不会被麦克风当成用户？这既是序列建模问题（别把自己的输出当输入条件搞混），也是声学前端问题（回声、泄漏、混响）。双流架构解决前者，不等于自动解决后者。

端到端 vs 可控。 一体化多流模型联合优化潜力大，但难调试、难复用已有强半双工模型。外挂控制器可插拔、可审计，但控制与内容割裂，内容上限仍由后端决定。未来更可能是混合：模型负责理解世界与产生行为，系统负责实时性、安全与确定性。

这三场仗解释了为什么领域不是一个模型取代上一个模型，而是同一问题的不同下注。

2.4 再补一层坐标系：双工决策落在哪一层

只说端到端还是级联仍然不够细。更有用的追问是：**系统决定现在该听还是该说的那个决策点，落在模型栈的哪一层？** 这一点可以收成 L0–L3，补上前面二维表缺的决策落点轴。

层级	决策怎么做	常见可达状态	代表形态	代价 / 备注
L0	模块外挂：VAD / EoT / 对话管理器在 LLM 外面跑	Idle/Listen/Speak 清楚；Wait 常退化；Dual 往往到不了	FireRedChat、FlexDuo 一类可插拔控制器	决策可以很快，但决策→大模型→首包语音仍有延迟地板
L1	隐状态旁路：从 LLM 隐状态读出，旁路预测器决定听/说或另开语音头	可显式 Wait；Dual 常只是有限/退化支持	MinMo / Freeze-Omni 等；许多 Thinker–Talker / 双脑流式系统（同形状未必宣称全双工）	L1 是跨产品目标反复出现的结构吸引子，不是某一团队的偶然发明
L2	词元/状态同步：听与说写进同一条（或并行）可监督的 token 时间线	Idle/Listen/Speak/Wait/Dual 在构造上都可以到	Moshi 并行多流；OmniFlatten；SyncLLM 块交替；StepAudio 3 的 320ms 状态交错	同属 L2，子设计差很远——并行流、展平、块交替、状态词元不是一家
L3	共享连续潜表示：双工决策沉到统一表征，不靠离散词元谈判 Dual	理论上 Dual 可成为自然状态	公开系统基本空白	卡在生态（离散 codec 默认）、流式连续生成未熟、形式定义未定

还有三条容易踩坑的结论：

1. **L0 不是过时废物。** 2025 之后仍有严肃工作主张模块化在延迟、可解释、工程成本上长期有席位。L0–L3 是设计选择轴，不是进化等级。
2. **L1 形状 ≠ 全双工主张。** 同一隐状态旁路出语音可服务全双工，也可只服务流式文本+语音双模态。把 Dual-Brain 直接翻译成全双工，是概念滑坡。
3. **L2 打开 Dual，不保证会 Dual。** 架构给的是可达性；后训练数据覆盖哪些交互格，才决定行为实现度。看一个系统，先问决策在 L几，再问它真演示了哪些交互格。

2.5 交互本体：一个全双工时刻 = 时间 × 意图 × 响应

四问框架（听什么/何时说/说什么/怎么说）仍然对，但何时说太粗。更可操作的分解是把**每一个瞬间**写成三元组：

维度	含义	常见取值
T 时间关系	双方声音在时间上怎么叠	顺序交接 / 零间隙锁存 / 重叠 / 持续并发 / 双方静音
I 用户意图	用户这一声在干什么	给信息 / 附和 / 纠错 / 抢地板 / 让地板 / 不流畅犹豫 / 第三方声
R 系统响应	系统当场该怎么动	继续说 / 停下 / 等待 / 自己也附和 / 忽略 / 开口接话

三维正交。把它们糊成一句支持打断，会把完全不同的失败混在一起。笛卡尔积理论上很多格，真正物理上可能、产品上致命的并不多。实践里最致命的，常落在下面六格酸测——几乎每格都能对上交互四轴与真实工程惨案：

酸测格	人话	常见失败
标准接话	用户说完，系统干净接上	人人都会；只测这个等于没测双工
零间隙锁存	几乎无缝接上，系统要在用户说完前已开始生成	固定静音门限永远慢半拍
合作式打断	用户抢地板，系统约 200ms 内让路	伪双工常按块边界才停，听感像没听见
说话中附和	用户"嗯嗯"，系统必须继续说	最常见用户抱怨：把附和当打断，话说到一半被自己掐死
第三方/噪声	重叠声不是对你说的，应忽略	无说话人条件就容易乱让路或乱接话
长静音犹豫	声学静了但语义未完，应等待	VAD 五百毫秒一到就抢，Pause 轴崩盘

还有一格几乎所有公开系统都交白卷：**持续并发（双方长时间同时说话且系统仍正确）**。架构上有的 L2 能表示，训练与评测上普遍缺席。它是数据与评测的共同空洞，也是后面要谈的实现裂缝的极限案例。

这张本体表有三重用途：切训练数据、对齐评测、做消融库存。说这个系统不是总是全双工没有信息量；说它在说话中附和格失败才能指导改模块或补数据。Pause / Turn / Backchannel / Interruption，可以看作酸测格在工程评测里的压缩投影。

2.6 五态十一转移：系统此刻在干什么

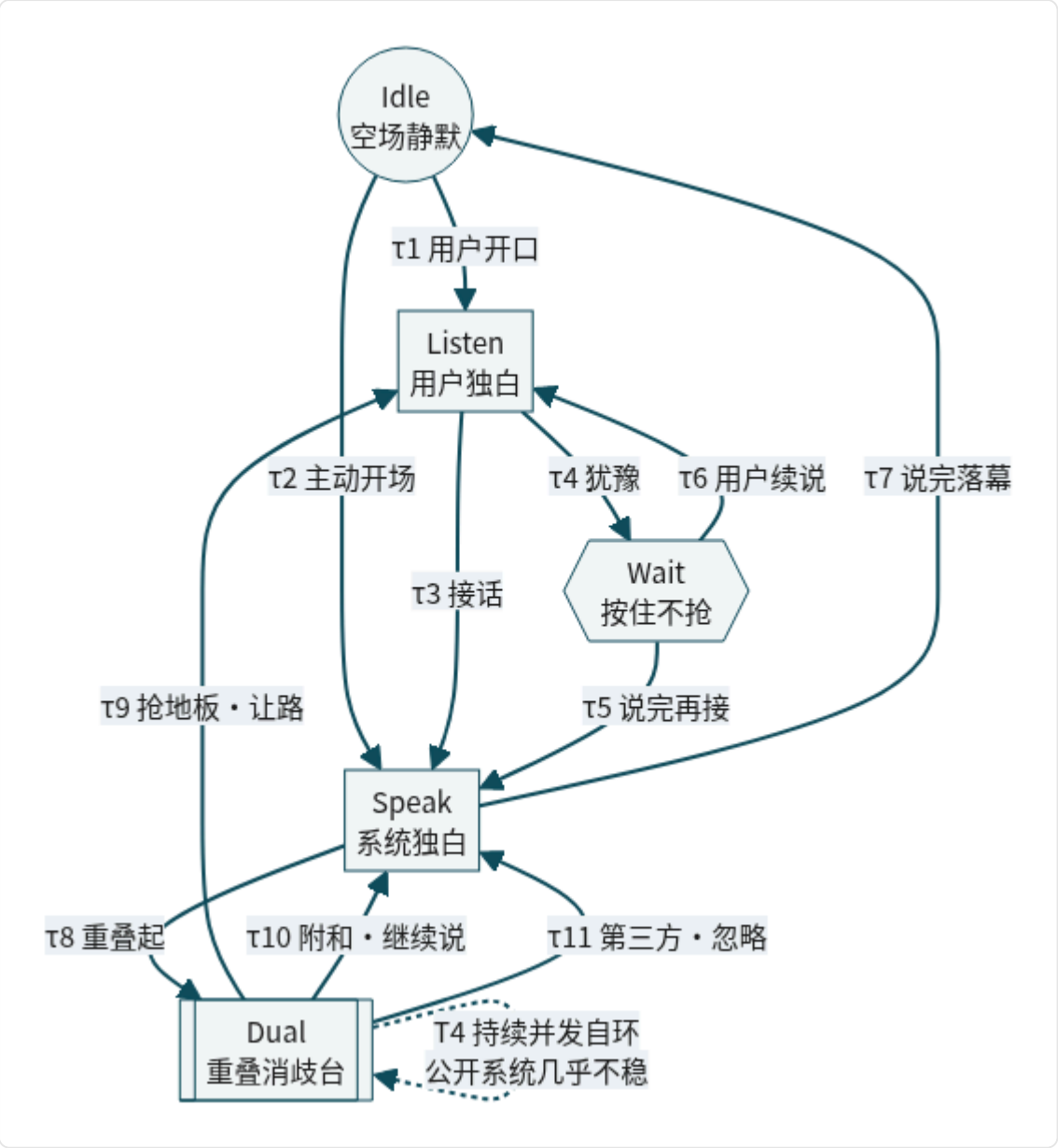
层级回答决策在哪算；本体回答面对何种交互；还缺一张图回答：**系统此刻处于什么行为态**。二值 Speak/Listen 太粗——真人对话不是两盏灯互斥开关，而是一场有灯光调度的舞台。

把对话想成一张舞台。聚光灯可以打在用户、打在系统、两边同时亮、也可以故意空场；再加一盏「犹豫灯」：声学上静了，语义上没完。五态就是五种灯光布置：

状态	舞台上发生了什么	系统此刻在干什么
Idle	空场：双方都不说，或系统主动忽略无关输入后的静默区	不响应、不写脏上下文
Listen	用户独白，系统侧幕静听	听、攒语义
Speak	系统独白，用户侧幕静听	出声，同时监测是否被打断
Wait	用户台词卡住了，系统按住不抢场	语义未完就忍，不把停顿当话轮结束
Dual	两边同时出声：重叠台上必须当场消歧	判意图后三选一：让路 / 继续 / 忽略

注意：FlexDuo 的 Idle 更偏助手说完后的缓冲过滤态，与会话尚未开始的经典 Idle 功能位置不同，但思想同源——把「不响应」从 Listen 的失败模式里拆出来。Wait 则专门承接犹豫酸测；没有 Wait 的系统，常把犹豫塌缩成「延迟了的接话」——指标上看起来像慢，状态上却少了一格可监督行为。

十一类转移，就是这五种灯光之间的换场指令。综述里把它们编号为 $\tau_1 \cdots \tau_{11}$，并分成三族：**起场族、交棒族、重叠族**。整张状态机如下：



五态十一转移：起场 / 交棒 / 重叠三族；虚线为公开系统几乎不稳的 Dual 自环

十一转移清单：每一跳在干什么

#	转移	家族	触发直觉	系统动作	一句话戏感
τ_1	Idle → Listen	起场	用户开口（VAD / 说话人条件 VAD）	感知态切换，未必立刻出声	空场亮用户灯
τ_2	Idle → Speak	起场	静默超时 / 主动开场	主动发起（R6）	空场亮系统灯：我先开口
τ_3	Listen → Speak	交棒	话轮结束确认（EoT）	接话回应（R6）	用户落幕，系统接棒
τ_4	Listen → Wait	交棒	声学停了，语义未完（犹豫）	等待（R3）	用户卡词，系统按住不抢
τ_5	Wait → Speak	交棒	犹豫后语义真正结束	再接话（R6）	忍够了，确认说完再开口
τ_6	Wait → Listen	交棒	犹豫后用户又继续说	感知态切回听	用户续上了，系统继续侧幕
τ_7	Speak → Idle	交棒	系统自己说完	感知态落幕	系统落幕，回到空场
τ_8	Speak → Dual	重叠	系统在说时用户侧有声	进入消歧台（意图未判）	两边同时亮灯——先别急着停
τ_9	Dual → Listen	重叠	判为抢地板 / 真打断	让路停说（R2）	对方抢戏，系统退场去听
τ_{10}	Dual → Speak	重叠	判为附和（嗯、对、好）	继续说（R1）	对方只是点头，系统把戏演完
τ_{11}	Dual → Speak	重叠	判为第三方 / 噪声	忽略（R5）	隔壁桌说话，系统当没听见

其中 τ_1、τ_6、τ_7、τ_8 多为**感知转移**：先改状态，不一定立刻生成新话；τ_2、τ_3、τ_5、τ_9、τ_{10}、τ_{11} 带着明确的响应策略。还有一格不算进「十一」却常被提起的：**Dual 自环（持续并发 T4）**——两边长时间同时说话且系统仍正确整合；公开系统几乎进不稳，综述因此把它从行计数里拿掉，却仍标成极限空洞。

起场族（τ_1–τ_2）：戏怎么开

对话从空场开始。用户先开口，走 τ_1；系统主动寒暄或超时提醒，走 τ_2。半双工与全双工在这里几乎无差别——难的不是开场，是开场之后还能不能处理重叠。

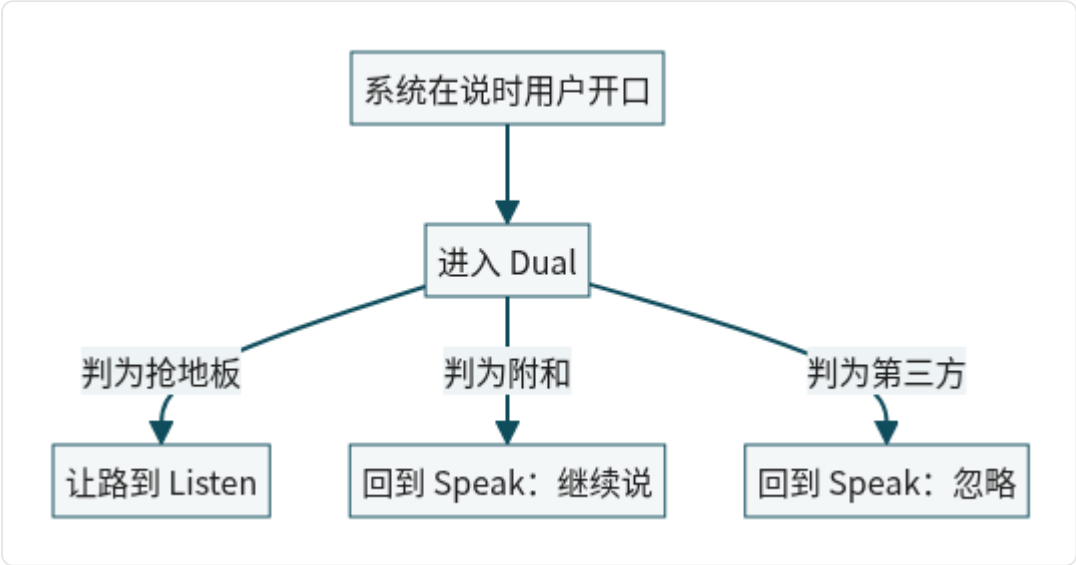
交棒族（τ₃–τ₇）：干净轮次的全部家当

标准问答几乎只靠这一族：Listen → Speak（τ₃）→ Idle（τ₇）。犹豫酸测多出一条岔路：Listen → Wait（τ₄），然后要么 Wait → Speak（τ₅）接话，要么 Wait → Listen（τ₆）用户续说。

没有 Wait 的系统，常把 τ₄ 塌成「Listen 里干等再 Speak」：体感像慢，状态上却不可监督——你分不清它是在忍，还是 EoT 坏了、算力卡了。Wait 的价值，是把「会忍」写成一格可测行为，而不是把犹豫伪装成延迟。

重叠族（τ₈–τ₁₁）：半双工与全双工的分界线

真正区分半双工与全双工的，是重叠族。系统在说（Speak）时用户侧突然有声，先走 τ₈ 进 Dual——这是消歧台，不是判决书。然后必须按意图分叉：



重叠族：同一入口 τ₈，三条出口 τ₉ / τ₁₀ / τ₁₁

Dual 出口	意图	正确策略	半双工常犯的错
τ₉ Dual → Listen	抢地板 / 真打断	立刻让路，改听用户	——半双工只会这一条
τ₁₀ Dual → Speak	附和（嗯、对）	吸收短反馈，继续说完	把嗯当成打断，中途掐断自己
τ₁₁ Dual → Speak	第三方 / 噪声	忽略，继续说	把旁人声当用户，乱让路或乱接话

半双工的形式化病灶可以写得很硬：它把 τ₈ 直接做成「无条件快切 Listen」，**根本不区分 τ₉ / τ₁₀ / τ₁₁**。所以有 barge-in ≠ 全双工；能在 Dual 里做对消歧，才算。附和与第三方同形不同因——轨迹都是 Speak → Dual → Speak，差在意图分类（继续 vs 忽略）。

几条完整剧本：从轨迹读系统

场景	状态轨迹	用到哪些 τ	要点
标准半双工问答	Idle → Listen → Speak → Idle	$\tau_1 \to \tau_3 \to \tau_7$	无重叠消歧；人人都会
犹豫后接话	Listen → Wait → Speak	$\tau_4 \to \tau_5$	有 Wait 才叫会忍；否则只是 Listen 里干等
犹豫后用户续说	Listen → Wait → Listen → Speak	$\tau_4 \to \tau_6 \to \tau_3$	证明 Wait 不是死胡同
Moshi 式合作打断	Speak → Dual → Listen	$\tau_8 \to \tau_9$	让路 ≈ 抑制己方出声，用户流继续编码
说话中附和（最难、最常挂）	Speak → Dual → Speak	$\tau_8 \to \tau_{10}$	缺这条，就会把嗯当成打断
第三方拒识（FireRedChat pVAD 一类）	Speak → Dual → Speak（忽略）	$\tau_8 \to \tau_{11}$	与附和同形不同因
持续并发（公开系统基本没有）	Speak → Dual → Dual → …	τ_8 + Dual 自环	数据与评测两侧都薄

状态机**不是控制器说明书**。它描述外部可观察行为；算转移的可以是 L0 模块、L1 旁路、L2 解码——状态机保持沉默，只问「你此刻在哪一态、走了哪一跳」。它也不是难度均匀的清单：τ_3 / τ_7（干净接话）几乎人人会；τ_{10}（附和继续）、τ_{11}（第三方忽略）、Dual 自环才是硬骨头。缺某一跳，就缺所有依赖它的酸测格——这正是状态机当消融库存的用法。

2.7 实现裂缝：架构天花板 × 数据覆盖 × 评测分数

把三层框架叠在一起，会得到一套很实用的诊断式：

诊断层	看什么	回答的问题
架构 L 层	理论上能到达哪些态 / 格	天花板在哪
训练数据	实际覆盖了哪些格	实现了多少
评测分数	演示出了哪些格	报告了什么

三者不一致，就是**实现裂缝（realization gap）**。两个同属 L2 展平族的系统，一个能展示附和与并发主张，一个在基准上几乎看不到——差别往往不在网络拓扑口号，而在后训练数据覆盖了哪些交互格。Moshi 公开把 Fisher 真双通道微调当作话轮能力关键来源；有的 L1 预测器本身很浅，却靠数千小时带话轮标签的对话混料跑出可用的打断/附和行为。结论很清醒：

> 在 L0–L2 范围内，束缚系统级全双工行为的，常常不是有没有选对 L 层，而是 **Type C 双通道时间对齐对话数据** 的细胞覆盖分布。

数据可以粗分成三类：

类型	是什么	教什么	教不了什么
A 单流语音	ASR 式语料	语音词表/对齐	交互动力学
B 单流对话/指令	语音问答、指令	用声音回答	重叠与附和时机
C 双流时间同步对话	双通道、帧级对齐	真正的全双工行为	——这是绑定约束

公开 Type C 资源长期被电话时代英语语料主导，总量相对工业专有数据是不对称的；合成与自动标注能放大规模，也会带来分布偏见。酸测里长期缺席的持续并发，在公开训练与公开评测两侧同时空洞——即使有人明天靠专有数据做出来，也缺少干净的公共尺子去证明。

评测侧还有命名陷阱：Full-Duplex-Bench 系列与另一套 FD-Bench 管道式工具不是同一件事；v1.5 把重叠拆成打断/附和/对他人说/背景声，才真正对齐本体的重叠行；v2 变成多轮主动考官；v3 进一步压到工具与真实不流畅。读分数时要问：这是在测标准接话（人人会），还是在测附和消歧与第三方拒识?

对工程报告，综述式建议是三件套一起写，而不是只丢一个总分：**(i) 决策在 L几；(ii) 训练数据覆盖哪些格；(iii) 基准上演示了哪些格**。 单一综合分会把架构不够 / 数据不够 / 尺子偏了糊成一句双工不行。

2.8 把尺子收成一张诊断表

如果说全双工的本质是连续时间上的状态演化，那前面几节其实在回答四件更具体的事：

要问的事	对应尺子	它防止你混淆什么
状态演化写在哪一层?	L0 / L1 / L2 / L3	别把外挂控制器和原生多流当成同一种端到端双工
演化要面对哪些事件?	T×I×R 酸测格	别用一句支持打断糊掉附和、第三方、犹豫
演化此刻处于哪一态?	Idle / Listen / Speak / Wait / Dual	别把二值听/说当成完整行为空间
演化有没有被训练出来?	实现裂缝 = 天花板 − 数据覆盖	别用架构口号替代 Type C 数据与评测演示

Moshi、FlexDuo、StepAudio、交互 RL、MiniCPM-o，都可以用这张表对照，而不必陷入模型名词竞赛。关键是：**打开 Dual 的架构，若训练里没有附和/第三方/并发格，行为仍会塌回一有**

重叠就让路的半双工策略。

2.9 再加一层：全双工实现出来有哪些形态，功能化后又如何逼出不同算法

尺子还不够落地。工程与产品里，**全双工并不是一种实现，而是一族表现形式**：同一标签下，系统在实现什么功能、服务什么应用、把决策写进哪段算法上可以差得很远。这一节把状态演化拆成可观察的形态与功能点，免得后面看具体模型时，误以为大家都在解同一道题。

2.9.1 表现形式：从能打断到连续多流状态演化

历史上与工程上，人们口头说的全双工，至少叠着五层越来越强的主张。它们可以共存，但**不应混称为同一种能力**：

形态	用户看到什么	典型手段	算法重心	未解决 / 新问题
F1 可打断的半双工	播报中能停；本质仍是说完再答+前端开关	VAD / 关键词停播 / turn_detection	端点检测、播放打断、会话复位	重叠建模、附和消歧、边说边改内容
F2 系统级全双工	外挂控制器让半双工后端用起来像双工	pVAD + EoT + FSM（含 Idle）；L0	状态转移、上下文过滤、控制与生成解耦	Dual 内消歧弱；内容上限绑死后端；链路延迟地板
F3 原生生成式双工	说话时仍持续编码用户；重叠写进生成分布	用户 // 助手多流；或块级双流 + 状态词元；L2	时间对齐、多流掩码、静音/开口联合分布	Pause/BC 易过激；无 Idle；角色/语言/工具另欠账
F4 双工 + Think/Act	地板之外还能边想边说、异步调工具	状态交错 + TWS + Voice Agent	听/说/想/行多时间线调度、路由、异步回写	路由次优；实时指令遵循可能掉；工具长程约束难
F5 世界/多模态双工	对象从用户话轮升为持续视听环境；可主动开口	时间窗局部更新、听/说控制与内容解耦、播放对齐	多模态序列化、窗长、TAIL 类对齐	短事件分辨率、主动打扰成本、端侧算力

还有一条**横切增强线**，本身不是新的 F 形态，却常被误当成下一代双工：

横切	改什么	不改什么
C1 条件化（人设/音色前缀）	像谁	地板算法族
C2 跨语/跨脚本迁移	文本流符号系统与 Type C 数据	双工时间模型本身
C3 交互后对齐（四轴 RL 等）	已有 L2 上的实现度	决策落在哪一层
C4 感知/推理底座	为 F4 准备弹药	本身仍可能半双工部署

演化也不是F1→F5 逐级淘汰，而更像分叉：

路径	做什么	典型场景
F1 → F2	用可控补丁升级存量半双工	工业快车道、要可插拔与拒识
F1 → F3	用多流/状态词元重写时间模型	研究与高端体验、要重叠建模
F3 → F4	叠思维与行动时间线	推理顾问、办事型 Voice Agent
F3 → F5	把对象换成世界时间轴	看护、带屏现场助手
F3/F4 + C1/C2/C3	横向补角色、语言、四轴实现度	多角色客服、跨语、timing 对齐

真正的范式转移仍是 turn-based → continuous multistream agent；但到达 agent 之前，**系统派与原生派可以并行**，能力底座也可以先于双工核成熟。

2.9.2 功能化：同一全双工标签下，到底在卖哪几项能力

搭出来之后，功能不会均匀长齐。更有用的是按**功能点**切开——这正是交互四轴与酸测格的工程翻译，再向外扩到应用与 Agent：

功能点	用户感受到什么	主要机制落点	应用更敏感场景
地板：Pause	犹豫时不被抢	Wait / 语义 EoT / 少开口奖励	客服长诉说、老人慢语
地板：Turn	说完立刻被接住	接话转移、低延迟开口	语音助手问答
地板：Backchannel	讲述中有"嗯"且不抢话题	Dual 内继续说分支	陪伴闲聊、倾诉
地板：Interruption	中途改口，系统让路并接上	Dual 内让路+ 内容重答	导航改目的地、纠错
拒识：第三方/噪声	旁人说话不乱接	Idle / pVAD / background rejection	门店、车载、开放办公
条件：人设音色	像某个角色、某副嗓音	混合前缀 / 参考音频	多角色客服、个性化助手
迁移：语言脚本	非英语也有自然节奏	词表重 init + 真双通道	区域市场、code-mix
认知：边想边说	既快又不太蠢	TWS、自适应路由、MTP	复杂咨询、需要推理的口语
行动：异步工具	办事时对话不卡住	Voice Agent、结果回写	订票、查账、工作流
世界：视听主动	看见变化会提醒	Omni-Flow、主动策略	陪伴看护、现场助手
效率：端侧实时	能在设备上跑	稀疏注意力、量化、文本慢/语音快分工	手机/嵌入式

关键在于：功能点互相打架，会直接改写架构偏好。

- 强调 Pause + 拒识 → 更偏向显式 Idle/Wait、语义终点、保守策略（F2 或 F3+后对齐）。
- 强调 Turn + Interruption 极限低延迟 → 更偏向 F3 细帧多流或高节拍状态机，并接受 Pause 变差的风险。
- 强调 Backchannel → 必须有 Dual 内继续分支；纯 F1/F2 半双工后端几乎天生做不好助手侧短附和。
- 强调边想边说 → 必须引入第二条时间线，否则又退回先想完再念的半双工智商税。
- 强调工具 → 异步与共享上下文成为一等公民，否则双工在调 API 时死亡。
- 强调世界主动 → 用户不再是特权会话角色，窗更新与打扰成本进入目标函数。

于是同一句我们做了全双工，至少可能指五档完全不同的能力：

理解	实际含义	大致停在
A	能 barge-in	F1 / F2
B	重叠可建模	F3 构造
C	四轴酸测都像人	F3 + 数据/对齐实现度
D	听 // 说 // 想 // 行	F4
E	与世界同轴演化	F5

后面讨论默认以 **C 为验收下限，D/E 为演化方向**；A/B 在工程上完全合法，但别用 F1 的打断成功率去证明 F4 的边想边说。

2.9.3 不同理解如何逼出不同的算法与架构设计

下面按你选择了哪种理解 / 形态，反推**算法与架构必然长什么样**。这是本层最硬的部分：表现形式不是营销话术，它会改损失函数、序列布局、训练日程与推理循环。

若目标是 F2（系统级双工）

- 架构：控制与内容解耦；控制器吃窗口音频+历史，吐动作；后端可换。
- 算法：多类状态分类 / 转移学习；类别极不均衡（继续听远多于打断）；必须注入偏好偏置（如接话严格在结束后、听起点延后以区分附和）。
- 数据：双通道上标动作，不必重训整个语音 LLM。
- 代价：助手附和弱；Dual 构造不足；两模型串联延迟。

若目标是 F3（原生生成式双工）

- 架构：并行用户/助手表示（多码本同拍，或连续编码双波形流）；取消显式 turn 边界。
- 算法：联合自回归或块级状态词元；声学 delay / 深度小网络解决 K tokens/frame；文本脚手架必须按帧而非整段。
- 训练：单流 → 伪双流格式 → 真双通道动力学 → 指令；缺真 Type C 则重叠行为虚。
- 代价：每帧大算力或块分辨率权衡；Pause/BC 需额外对齐；自听仍要声学前端。

若目标是 F4（双工 + 想/行）

- 架构：在 F3 之上加私有推理流与工具状态；同模并发或双脑调度。
- 算法：播放感知放行、Speak-First/Think-First、自适应是否思考、对说出口路径严格校验、工具异步与槽位澄清。
- 训练：先能力与接地推理，再双工地板，再 Agent 轨迹；合并多教师能力。
- 代价：路由失败会伤推理维；实时指令遵循可能掉；工程调度复杂。

若目标是 F5（世界双工）

- 架构：共享时间轴上的视听+输出窗；LLM 宜慢文本通路，语音快通路。
- 算法：先感知后输出的窗内顺序；LS 控制与内容解耦；按播放进度限制新文本（防说旧话）。
- 训练：带时间戳的全双工多模态样本；主动行为与应答样本并重。
- 代价：窗长甜区；短打断分辨率；端侧与云端形态分叉。

若目标是横切条件化 / 迁移 / 后对齐

- 条件化：最小充分改动是时间前缀，而不是重造 codec；损失屏蔽提示段。
- 跨脚本：词表相关参数重 init + 冻 codec + 真双通道两阶段——交互智能与语义智能会脱钩暴露。
- 后对齐：奖励看波形、梯度走控时机的文本流；四轴联合采样；必须加语义奖励防为快而胡说。

把常见工作钉在形态 × 功能上（仍然服从可达 ≠ 已实现）：

模型 / 工作	主形态	主攻功能点	对算法 / 架构的硬要求
Moshi	F3	重叠建模、低延迟接话/打断	多流 RQ、Inner Monologue、Type C
FlexDuo	F2	Idle 拒识、可插拔地板	FSM、上下文提交、动作不均衡学习
FireRedChat	F2	pVAD 第三方、EoT 犹豫	个性化 VAD + 语义终点
Step-Audio 2	C4	理解/生成/工具（双工核未立）	latent 入、交错出、检索
接地推理	C4	音频真推理	模态接地蒸馏，否则 TWS 有害
StepAudio 3	F3+F4	地板四轴 + 边想边说 + 异步工具	状态交错、TWS、Agent 回写
PersonaPlex	F3+C1	角色+音色	Hybrid 前缀、合成剧本
Human-1	F3+C2	跨语节奏	词表重 init、真双通道
交互四轴 RL	F3+C3	Pause/Turn/BC/Intr 实现度	轴奖励 + 语义保底 + 课程上下文
MiniCPM-o	F5	视听同轴、主动、端侧	Omni-Flow、TAIL、效率栈

2.9.4 应用选型：先选功能组合，再选形态，最后才选模型名

产品讨论若从模型名开始，几乎必错。更合理的顺序是：

步骤	先定什么	再定什么
1	场景敏感功能点	Pause / 拒识 / 工具…
2	需要的酸测格与五态转移	犹豫、附和、第三方…
3	形态：F2 或 F3（或 F3+F4/F5）	可控补丁还是原生多流
4	具体骨干与数据处方	Type C、合成、RL…

举例：

- **嘈杂门店收银助手**：第三方拒识 + 打断 > 附和漂亮 → F2（pVAD/Idle）往往比单纯刷 F3 接话分更值。
- **低延迟闲聊伴侣**：Turn + Interruption + 适度 BC → F3，并准备 C3 压 Pause 过激。
- **合规多角色客服**：Pause/BC + 人设一致 + 安全 → F2 或 F3，加 C1，交互数据慎用闲聊分布硬刷。
- **推理型口语顾问**：F3+F4；接受 Speak-First 的早期不完整表述风险，或提高 Think-First 比例。
- **现场看护/带屏助手**：F5；主动策略要单独做打扰成本，不然很双工变成很吵。

2.9.5 选型时真正该问的三件事

形态表和功能表最终要落到选型上。比起背模型名，更稳的是连问三句：

问题	为什么问	答偏了会怎样
这个场景最怕哪几格酸测失败?	决定你要 Wait、Dual 消歧，还是第三方拒识	门店系统去刷闲聊接话分，钱花错地方
决策落在 L几、形态停在 F几?	决定可控性、延迟地板、Dual 可达性	用 F1 分数讲 F4 故事，评审和上线都会穿帮
序列布局与损失被什么问题逼出来?	决定你该买多流、状态词元，还是外挂 FSM	把别人的模块当积木乱拼，四轴此消彼长

论文名只是索引；先答这三问，再打开具体系统。

三、发展总脉络：阶段解决什么，论文落在哪

有了尺子，可以把领域演化收成一条主脉络：每个阶段主要还哪张账单、长出什么形态、有哪些代表工作。后面按阶段下钻，而不是按发布时间堆模型。

3.1 主脉络

顺序	发生了什么	仍可能留下的坑
1	轮次级联管线先做出来	慢、丢副语言、假轮次
2	流式 / 端到端降延迟、绕文本	仍可能是半双工
3	音频成为可建模语言（codec / token LM）	只是地基，不是双工核
4	原生打开 Dual（多流取消假轮次）	Idle、角色、工具、四轴过激
5	分叉还债：外挂地板 / 能力闭环 / 横向条件化 / 世界对象 / 刷实现裂缝	并行而非逐级淘汰
6	连续多流 / 多模态状态智能体	统一目标与评测仍未闭合

真正的转变不是ASR→LLM→TTS 逐渐端到端，而是 **Turn-based Speech Pipeline → Continuous Multistream Agent**。

3.2 阶段—问题—形态—代表工作

阶段	要还的账单	主要形态	代表工作
S0 轮次级联	先做出来	F1 前史	工业 ASR→LLM→TTS
S1 流式与直接 S2S	降延迟、绕文本	仍常 Half-Duplex	流式级联；直接 S2S 前史
S2 音频语言化	波形无法当长序列	地基，非双工核	AudioLM 路线；语义/声学分层；K tokens/frame
S3 E2E Spoken LLM	弱化 ASR/TTS 瓶颈	仍可能 Half-Duplex	Mini-Omni 一类实时语音（作半双工警示）
S4 原生打开 Dual	假轮次；重叠建模	**F3** / L2	**Moshi**
S5a 外挂可控地板	难复用；无 Idle；噪声污染	**F2** / L0	**FlexDuo**；对照 FireRedChat
S5b 能力→实时闭环	听不深、想不真、办不了事、深思与延迟冲突	C4→**F3+F4**	**Step-Audio 2 → 接地推理 → StepAudio 3**
S5c 横向条件化	固定人设音色；跨脚本崩	F3+**C1/C2**	**PersonaPlex**；**Human-1**
S6 世界对象升级	Blocked-I/O；被动；端侧	**F5**	**MiniCPM-o 4.5**（效率底座 MiniCPM4）
S7 刷实现裂缝	似然不优四轴；Pause↔Turn 打架	F3+**C3**	**Multi-Faceted Interactivity Alignment**

这些阶段是并行还债，不是串行淘汰：S5a 与 S4 竞争又互补；S5b 的前两级火箭可以先于双工核成熟；S5c / S7 是横向增强。后面各章标题按同一语法写：**还哪张账 → 落在哪一形态/横切**，好跟第二节的 L / 酸测 / 五态 / F·C 尺子对上看。

3.3 每个阶段怎么拆

进入具体阶段时，固定看四件事：被什么问题逼出来；代表工作各自站哪一格（L几、F几、盖了哪些酸测格）；算法与架构如何回答；还清了什么、又新欠下什么。先从地基开始——它回答的是：还没有 Dual 之前，领域先还了什么账。

四、S0–S3：地基先行——让声音可建模，并守住「E2E ≠ 全双工」

4.0 阶段定位

项	内容
账单	波形太长无法建模；文本瓶颈；流式≠双工
形态	地基（尚非 F3）；E2E 语音模型仍可能 Half-Duplex
代表工作	AudioLM / codec 语义-声学分层（概念锚点）；E2E Spoken LLM（如 Mini-Omni 路线，作反例警示）
下一阶段	有了实时音频词表，才谈得上 Moshi 式原生双工

在谈双工之前，先谈一个更硬的约束：波形没法直接当长对话的 Transformer 序列。一秒钟大约四万多个采样点。若按采样点自回归，上下文长度与算力都会爆炸。

于是领域走了一条看似迂回、实则必要的路：

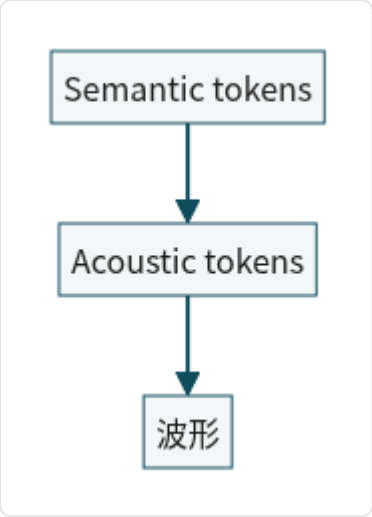


音频语言化闭环

这一步的意义，不亚于 NLP 里把字符变成子词。它让音频生成第一次可以借用语言模型的整套工具：自回归、上下文、采样、温度、后续还可以接强化学习。

但音频 token 不是铁板一块。同一句你好至少缠绕多层：语义（你好）、音素、韵律、说话人音色、情绪、频谱细节。若只用一套表示硬扛所有层，模型会陷入重建质量与长程结构之间的撕裂。经典做法因此分层：

层次	更偏什么
Semantic tokens	说什么、结构是否合理
Acoustic tokens	好听、像谁、什么情绪
波形	最终听感



语义 / 声学分层

这就是后来 Moshi 把层次再向上扩一层——先定字，再定语义声学——的思想源头。没有这层地基，所谓全双工 spoken LLM 根本无处落脚：你甚至没有一个能在实时帧率下运行的音频词汇表。

还有一个常被忽略的约束：**每一帧往往不是一个 token，而是 K 个码本 token**。若把 K 路在时间上展平，序列长度与算力直接乘 K，实时与长上下文一起崩。于是 L2 全双工的许多子设计——深度小网络同拍预测多码本、展平交错、按块打包——本质上都是在回答K tokens per frame 怎么排进自回归。它们看起来像 LLM 花活，其实是 tokenizer 上游约束逼出来的。

同时要记住：到了端到端语音大模型这一步，领域仍可能停留在半双工。模型可以从音频进、音频出，可以流式说话，但若没有并行听通道与重叠建模，它解决的是信息瓶颈与延迟，不是交互轴上的全双工。

五、S4：原生打开 Dual——多流取消假轮次（F3 / L2）

5.0 阶段定位

项	内容
账单	轮次假设失败；重叠/打断/附和无法被 turn 切分表达；级联延迟与文本瓶颈同时要压
形态	F3 原生生成式双工（L2）
代表论文	**Moshi**
还清	把边听边说写进联合自回归分布
新欠	Idle/角色/跨语/深思/工具/四轴过激

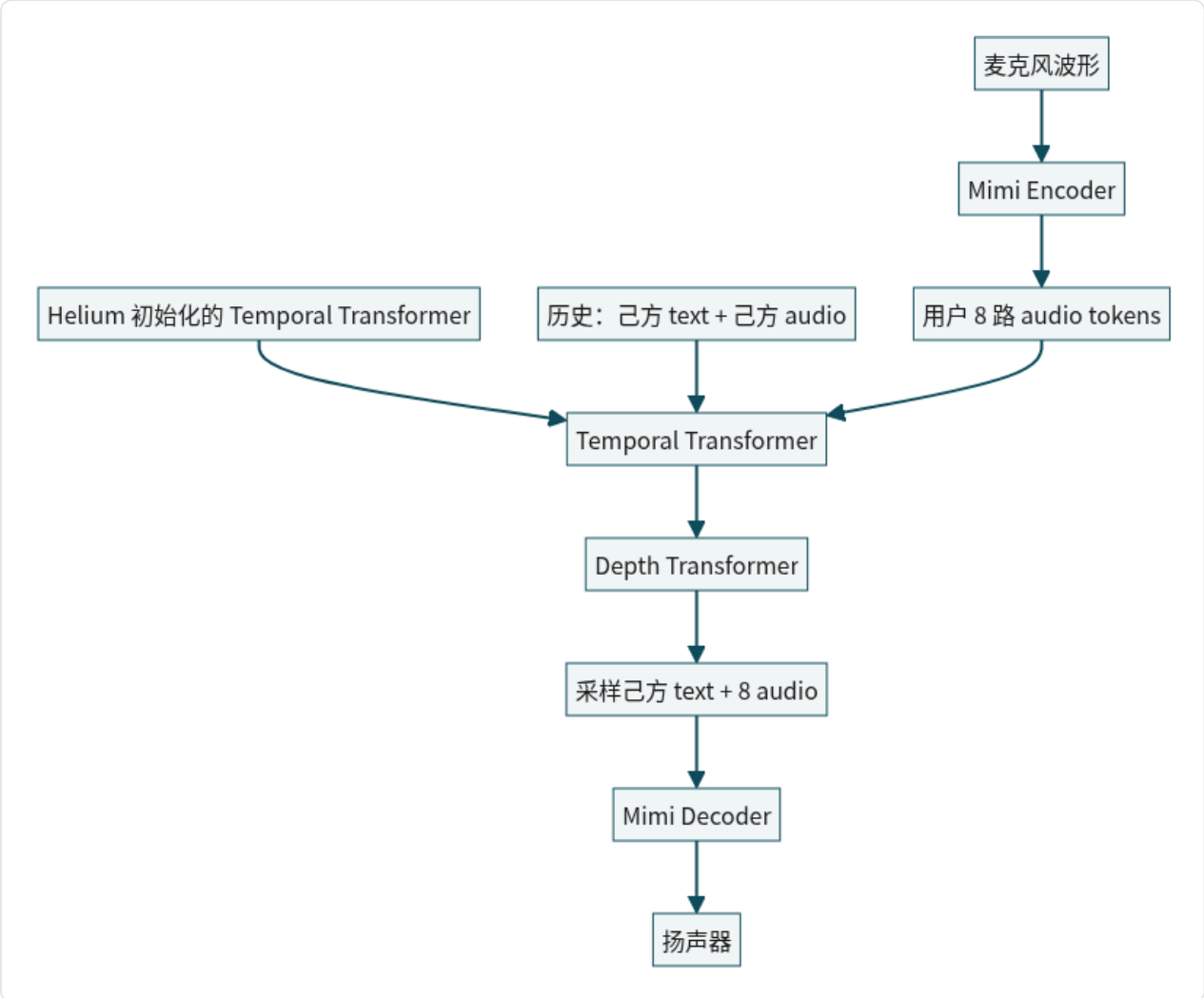
5.1 本阶段代表工作

工作	角色
Moshi	范式奠基：多流 + Mimi + RQ-Transformer + Inner Monologue
dGSLM 等前史	证明双流可学节奏，但缺实时文本 LLM 与声学 token 闭环（对照，不展开）

下面按模块深讲 Moshi——它不是又一个语音大模型，而是重新定义交互循环：系统永远在听，永远在产音频（说话或自然近静音）；用户与助手在同一时间栅格上并行存在。

5.2 总览：80 毫秒一拍的生命体征

把 Moshi 想成一台以 80 毫秒为心跳的机器。每一拍要完成：编码用户最新声音、更新内部状态、采样自己这一拍该说的字与声、解码播放。理论延迟大约是一帧加上一拍声学延迟，约 160 毫秒；实测大约 200 毫秒，进入人类自然回应量级。



Moshi：80ms 心跳推理循环

每一列约 80ms，联合大约 17 路：

通道	内容
己方 text	Inner Monologue 文本
己方 semantic	己方语义码本
己方 acoustic（delay）	己方声学码本（带延迟）
用户 semantic	用户语义码本
用户 acoustic（delay）	用户声学码本（带延迟）

下面按模块拆开。每一个模块都对应一个被逼出来的问题。

5.3 模块一：Helium——智能从哪里来

纯音频语言模型可以发声，甚至能续写一段像人的语音，但事实知识与推理通常很弱。语音对话若要能回答世界问题，不能指望从波形里长出维基百科。

Helium 是一个约 70 亿参数的文本 Transformer，用大量英文文本预训练。它的角色不是级联里的独立 LLM，而是把权重**初始化**给 Moshi 的时间维大模型 Temporal Transformer。训练 Moshi 时仍混入纯文本批次，防止音频训练把文本能力冲掉。

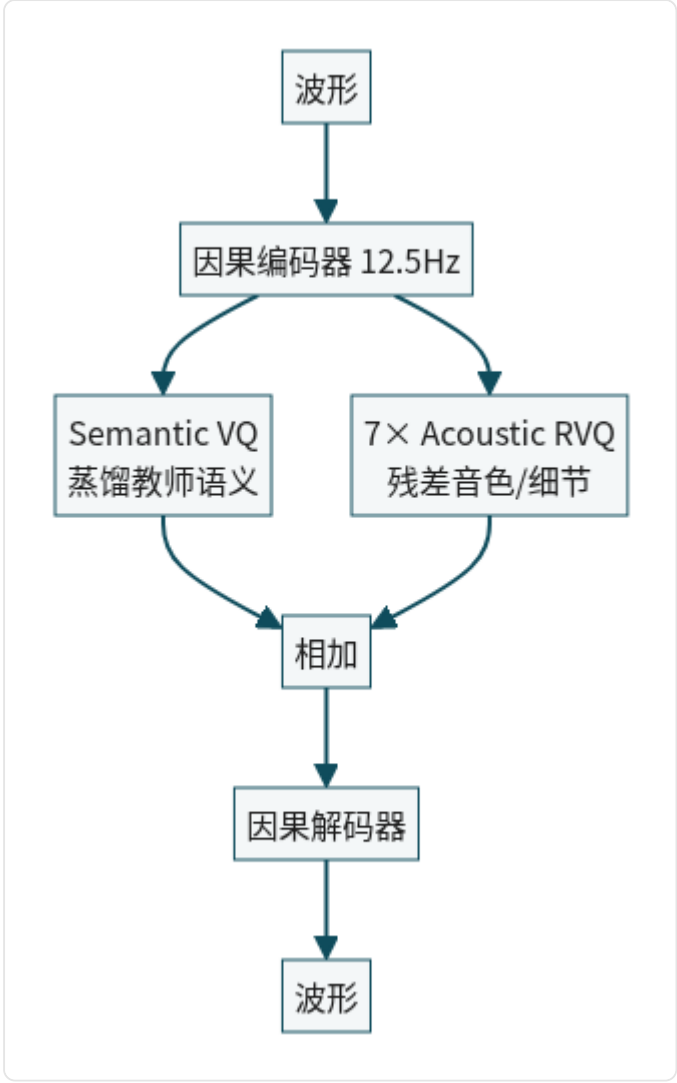
设计直觉很朴素却常被忽略：**口语系统的聪明主要来自文本预训练，副语言与交互动力学来自音频通路**。 两者必须同时保住。Helium 的局限同样清楚：英语中心的分词器、七十亿的知识上限、本身不含副语言。这些局限会在后来的跨语迁移、工具增强、角色控制里再次暴露。

5.4 模块二：Mimi——实时、低帧率、语义与声学拆开的 codec

Moshi 不能吃波形，必须吃离散 token。但普通神经 codec 若帧率太高，一秒钟几十上百个时间步，对话上下文撑不久；若只追求重建，又可能丢掉语义；若语义来自非因果的自监督模型，又无法实时。

Mimi 的选择是：

1. **因果卷积 + 因果 Transformer**，把 24kHz 波形压到约 **12.5Hz**，每步 80 毫秒，对流式友好。
2. **Split RVQ**：一路向量量化专门蒸馏语义（训练时用非因果教师，把语义烤进因果第一码本）；另一路七级残差量化专管声学。两路输出相加再解码，而不是让声学活在语义残差里——避免语义占满第一层、音质吃亏。
3. 总码率大约 1.1kbps，八个码本，每级词表规模可观。训练后 codec **冻结**，再训语言模型。



Mimi codec 结构

为什么合理？因为它同时满足三件互相拉扯的事：实时因果、足够低的帧率以覆盖数分钟对话、语义可懂与声学可听的折中。它也留下账单：码率上限约束清晰度；跨语种冻传可用但并非无损；用户侧也要实时编码，端侧算力吃紧。

5.5 模块三：RQ-Transformer——时间用大模型，深度用小模型

若把八个码本在时间上展平，等效大约每秒一百个 token，五分钟对话就是三万步，既难实时，也难长上下文。Moshi 因此采用残差量化 Transformer：

- **Temporal Transformer（大）**：沿时间走，吃上一时刻各路 embedding 之和，输出当前时刻隐状态。
- **Text head**：直接从 Temporal 出当前文本 token（不进 Depth）。
- **Depth Transformer（小）**：在同一时刻内部，自回归预测多个音频码本。

再配上 **Acoustic delay**：语义在当前拍，声学用上一拍或更早一拍。动机是降低同一时刻各码本强互依赖，让大 Temporal 承担跨层依赖，小 Depth 更好学。消融很硬：没有 delay，语言质量

差；delay 太长（例如像某些音乐生成里逐码本错开很多步），质量可以，但延迟对话不可用。对话场景的正解是：**低 delay + RQ 层次**，而不是无限并行或无限串行。

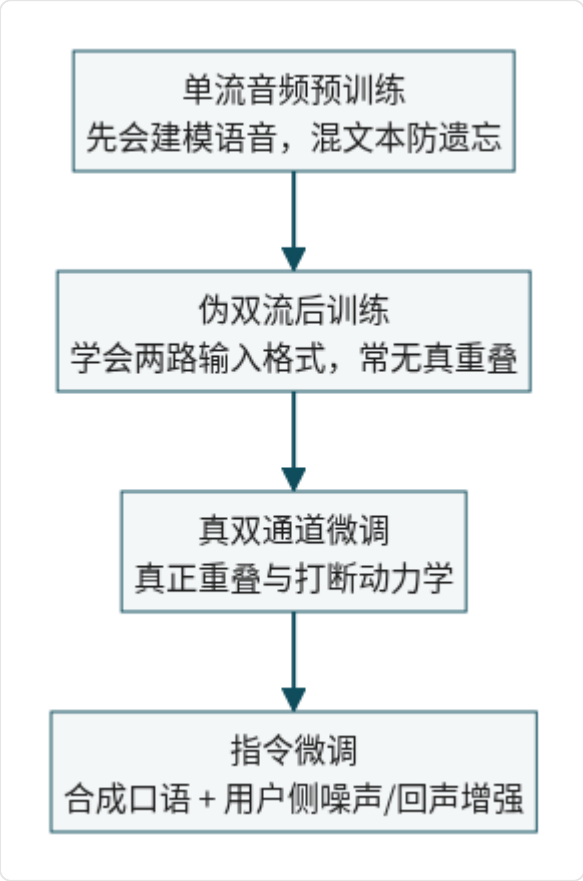
损失设计同样暴露优先级：文本与音频大致各半；语义码本权重远高于声学码本——先逼模型说对，再好听。PAD 文本降权，因为英文对齐序列里大量是 PAD。

推理时每 80 毫秒：用户新音频写入用户子序列（不采样用户侧），Temporal 前向，采样己方文本，Depth 采样己方音频，解码播放。若保持沉默，音频是自然近静音，文本多为 PAD；强制采到可开口的文本控制符，可以立刻打破沉默。这是隐式话轮控制的把手。

5.6 模块四：Multi-stream——取消显式 turn

单流把双方拼成一条序列，重叠难表示，必须显式切 turn，打断和附和难学。Moshi 在同一时间步拼接**两路完整音频流**：用户与助手。概念上删除 speaker turn——任意时刻双方都可以 active。

训练必须分阶段，否则格式与动力学搅在一起学不会：



Moshi 训练分阶段

关键思想是：**先模拟后真实**。伪双流解决模型认得两路输入，真双通道解决重叠时该怎么表现。指令阶段对用户流做回声、噪声、混响增强，是为了逼近产品条件，而不是刷干净实验室指标。

局限同样尖锐：伪双流阶段没有重叠，真双通道若不够，重叠行为会弱；没有显式 Idle，背景人声可能当作用户；固定一侧助手身份，难换角色音色；电话域声学偏窄。这些不是小瑕疵，而是后面整条分叉的种子。

用五态语言重说一遍 Moshi：它在构造上几乎一直潜在处于 Dual——用户流与助手流同拍存在；合作式打断是抑制己方出声、继续编码用户；附和消歧与第三方拒识并没有单独的显式转移头，主要靠数据分布撞出来。所以它在酸测上常表现为：接话/打断通道强，Pause 与 Backchannel 容易过激——实现裂缝在数据与目标函数，不在有没有双流。

还要纠正一句过强的话：双流**不等于**回声消除。双流避免的是序列层面把自己刚说的话当成用户输入；房间里扬声器漏进麦克风，仍要靠声学前端。

5.7 模块五：Inner Monologue——实时版先想字再发声

没有文本条件的音频 LM，可懂度与事实性容易塌，甚至缩成短促静音。另一极端是整段先写完答案再念——质量好，但必须等文本结束才能开口，与实时冲突。

Moshi 的折中是：只为**自己要说的话**建一条时间对齐的文本流。词开始处写入子词，中间填 PAD，词与词之间用边界符拆开结束上词与选下词。英文对话里 PAD 可以占到大约六成以上。在深度预测顺序里，文本是语义与声学的前缀：

顺序	每拍内部
1	text
2	己方 semantic
3	己方 acoustic
4	用户侧由真实输入填充（不采样）

消融显示：加入 Inner Monologue 后，转写负对数似然明显下降，生成长度恢复正常，口语问答准确率可接近数倍提升。它不是流水线中间模态，而是生成过程的内部脚手架。文本落后于音频时，同一架构可衍生流式识别；文本超前于音频时，可衍生流式合成。预训练阶段甚至随机抖动文本延迟，逼模型学会两种用法。

它不是深度推理链。复杂思考、工具调用、跨语言分词，都不在这个模块的职责里。它解决的是：**在实时约束下，如何把语言质量拴在音频生成上**。

5.8 Moshi 到底改变了什么

Moshi 把全双工从检测器开关推进到生成分布本身。系统不是先判断该不该说，再调用半双工生成器；而是每拍都在采样现在世界里双方声音的联合下一步。这是范式转折。

它同时把账单摊在桌上：固定人设音色、英语中心、七十亿知识上限、Pause/附和容易过激、背景拒识弱、生成中难以局部改口。后面每一条重要工作，几乎都在还这些账单中的一张或几张。

六、S5a：与原生派分叉——外挂 Idle/FSM，给存量后端装可控地板（F2 / L0）

6.0 阶段定位

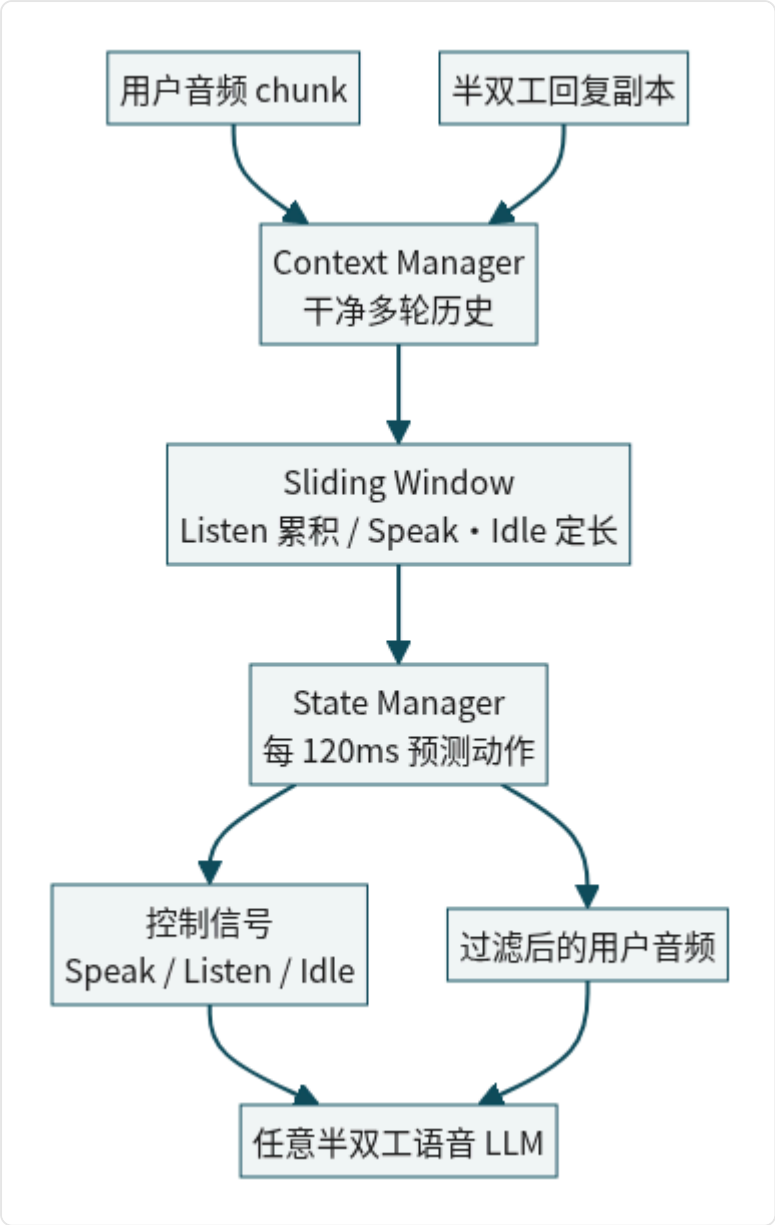
项	内容
账单	一体化双工难复用；无 Idle；噪声/第三方污染上下文；工程要渐进升级存量系统
形态	F2 系统级全双工（L0）
代表工作	**FlexDuo**；对照 **FireRedChat**（pVAD+EoT）
与 S4 关系	竞争又互补——工程可控 vs 联合优化天花板

6.1 本阶段代表工作

工作	角色
FlexDuo	Speak/Listen/**Idle** 七动作 FSM，挂任意半双工后端
FireRedChat	pVAD + 语义 EoT + 对话管理，完整可插拔系统（对照）

Moshi 的一体化很美，但工程会立刻遇到另一类痛苦：已有很强的半双工语音大模型，难道每次都要重训多流巨兽？噪声与第三方冲进上下文也不好局部修。本阶段的回答是：**全双工体验不一定等于端到端多流生成；它可以是半双工生成器加上高频话轮控制器**。

6.2 总结构



FlexDuo 控制流水线

工作流很直接：收音频、取历史、看滑动窗、预测动作、把控制信号和过滤音频送给后端、允许时生成、把回复写回上下文。切到 Idle 或 Listen 时可以打断后端输出；Idle 时用户音频不进对话历史；Listen 时才送去预填充。

6.3 为什么必须有 Idle

旧系统常是 Speak/Listen 二值。Listen 时环境声全进模型，上下文被污染，模型开始回答无关噪声里的幻觉内容，算力也在空转。VAD 能检人声，却难区分附和与有效话轮，又常靠固定静音阈值。

FlexDuo 把状态升为三态：

状态	含义	对系统的动作
Speak	助手在说	驱动输出，监测打断或说完
Listen	用户有效在说	音频累积并送后端
Idle	应忽略的音频	过滤，不写上下文

七类动作驱动状态机转移或保持：继续说、继续听、继续闲、说转听（被打断）、说转闲（说完）、听转说（接话）、闲转听（开始有效说话）。决策必须吃上一状态——这是转移，不是无记忆的帧分类。节拍 120 毫秒，接近人话轮决策粒度，又远粗于波形。

消融很能说明问题：去掉 Idle，接话综合指标下降，误打断上升。Idle 的双重作用是噪声过滤，以及语义完整缓冲——先忍一下，确认这是不是真要你接的话。

6.4 Sliding Window 与 Context Manager

Listen 时窗口累积，因为要完整语义；Speak 与 Idle 用固定短窗，降低算力。Context Manager 只在状态边界提交话轮：听结束才写入用户内容，说结束才写入助手内容。这样附和和噪声不会永久写进历史。条件困惑度可以因此大幅下降——不是模型变聪明了，是上下文变干净了。

6.5 这条路的边界

FlexDuo 抬的是交互控制与上下文纯度，不是内容智能上限。半双工底座若没为双工训练，助手很难自然产出短附和；两模型串联带来额外延迟与算力；它本质上是**可打断的半双工**，不是真正边说边根据新输入改口的连续生成。但它证明了一件对工业极其重要的事：控制器可以外置，全双工可以渐进升级存量系统。

与 Moshi 的关系不是谁淘汰谁，而是两条互补路线：

系统	交互怎么实现	层级	Dual	代价
Moshi	写进生成分布	L2	构造可达	联合优化天花板高，改动成本高
FlexDuo	写成可插拔策略	L0	往往到不了	可控可复用，内容与控制割裂

综述视角下，L0 的结构性天花板是：即使控制模块百毫秒级开火，后端大模型前向加首包合成，仍可能把从决策到开口推到约半秒量级。这不是否定 FlexDuo，而是解释它优化的是**交互正确性与上下文纯度**，不是 L2 那种把 Dual 写进生成分布的延迟形态。FlexDuo 的 Idle 更接近说完后的缓冲/过滤，对第三方与噪声格特别有用；对说话中附和仍继续说这类 Dual 内消歧，半双工后端天生吃力——因为后端本就不是为边说边听训练的。

七、S5b：能力先于双工核——听深、想真、能办事，再闭环实时地板（C4→F3+F4）

7.0 阶段定位

项	内容
账单	听不深、想不真、办不了事；深思与低延迟冲突；地板要显式可训
形态	横切 C4（能力/推理）→ F3+F4（实时双工闭环）
代表工作	**Step-Audio 2 → 模态接地推理（Step-Audio-R1 路线） → StepAudio 3 Realtime**
顺序纪律	2/R1 不是双工主节点；3 才把 Seamless Duplex 一等公民化

7.1 本阶段代表工作

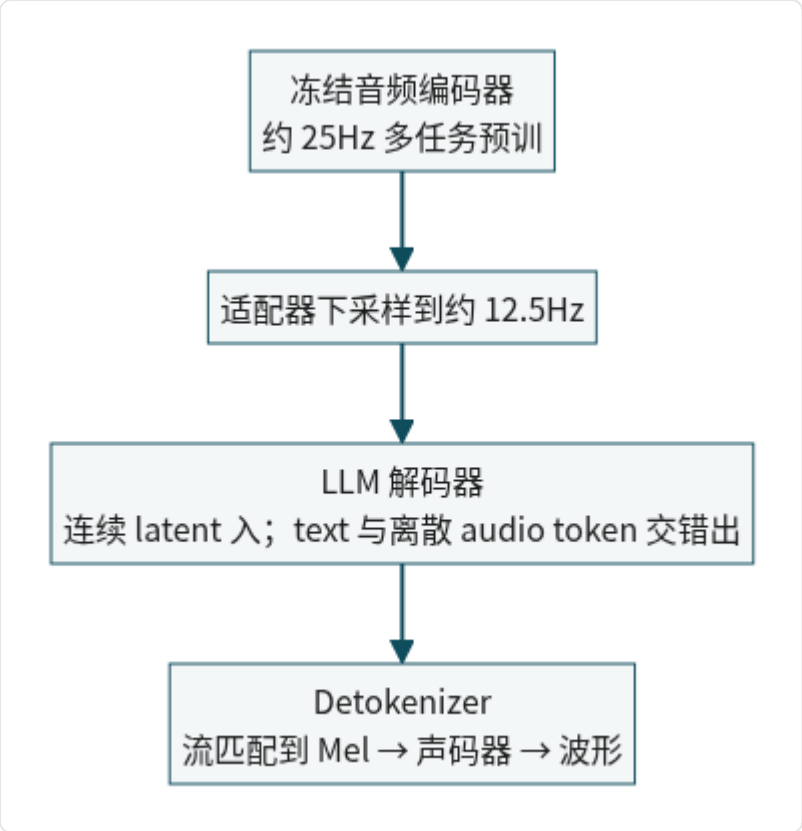
工作	阶段内角色	形态
Step-Audio 2	理解+生成+工具的工业底座	C4（部署仍常 VAD）
接地推理蒸馏	防止越想越错	C4
StepAudio 3 Realtime	状态双工 + TWS + Voice Agent	F3+F4 / L2

Moshi 证明能双工说话后，账单变成：听不深、想不真、办不了事、一深思就开口晚。本阶段不是再造一个更小的 Moshi，而是三层火箭——**先能力，再接地推理，最后才实时双工闭环**。顺序不能颠倒：没有想得对，边想边说只会更快输出错答案；没有地板管理，再强推理也只是半双工聪明人。与 Moshi 同属 L2 不同配方：Moshi 偏离散多码本联合自回归；Step 实时版偏连续编码进模型 + 块级状态可监督。

7.2 Step-Audio 2：理解、生成、工具统一，但还不是多流双工核

问题很具体：有的模型只对齐语义，丢掉副语言；有的能听懂情绪却只吐文本；要同时会说、会查、会选音色，很难。

架构分成四件套，关键是**输入与输出的表示故意分裂**：



Step-Audio 2 表示栈

为什么这样设计合理？输入侧连续，减轻一上来就量化丢掉的细粒度线索；输出侧离散，方便与文本共用自回归采样与训练栈；流匹配负责听感与音色，不必强迫语言模型直接预测高保真频谱。多轮靠历史预填充；工具与检索（含音频检索换音色风格）用来压幻觉、接世界知识。训练路径是大规模持续预训练 → 多任务监督微调 → 偏好优化与感知强化。

部署上仍常见 VAD 过滤输入。因此它在谱系里必须诚实标注为：**能力底座与准双工产品栈**，不是 Moshi 那种原生多流双工核。它推进的是What to hear / What to say / How to speak里的深度与工具，把When to speak的硬问题留给后一级火箭。

7.3 音频推理的陷阱：越想越错，除非接到声学上

一个反直觉现象：给音频模型加长思维链，成绩常常变差。原因不是不该思考，而是思维漂到了转写文本的幻觉上——模型在歌词提到悲伤这种语义层编故事，而不是在小调进行、下行旋律这种声学证据上推理。

解决思路是模态接地的推理蒸馏：先用任务思维链与可验证奖励做冷启动，再迭代挑选必须依赖声学才能答对的题，让模型自生成多条思维，过滤是否真正接地、逻辑是否成立、答案是否正确，再做多模态监督与强化，多轮把推理从文本幻觉拉回声学基底。

结论可以写得很硬：**推理能力可以从文本迁移，但若不蒸馏到声学，测试时多算力在音频域是有害归纳偏置**。后面边想边说若没有这一层，想得越久错得越稳。

这里要防止一名混淆：公开材料里还有双脑流式生成形态的实时变体，其结构更像隐状态旁路的 L1（想与说拆开），**不等于**已经解决全双工地板。真正把听、说、想、行收进连续闭环、并用全双工基准验收的，是下一级实时系统。

7.4 StepAudio 3 Realtime：把时间闭环拆成可并行的四段算法

到实时版，目标变成四段闭环：

模块	问题
Deep Perception	听懂什么
Seamless Duplex	地板归谁
Think-While-Speaking	想与说如何并行
Voice Agent	事怎么办

骨干是混合专家语言模型；音频前端是连续编码器加适配器；**用户音频与模型自己的音频双流**进入解码器，生成器输出回流到模型流——继承边听边说的并行条件，但不再绑死在单一 codec 八码本深度自回归。文本另有通路；口语生成器增量出声，并允许停顿、犹豫等表达线索。

训练也服从先能力、再交互：三阶段预训练固定较长上下文与海量音文混合，并抬高纯文本比例保住推理；中期训练把上下文拉长，大增音频理解与智能体交互，并为双工地板铺表示；再分叉出转写专精与实时交互，后接对话/推理数据、双工后训练、自适应思考监督、多教师参数合并，以及语音智能体轨迹。

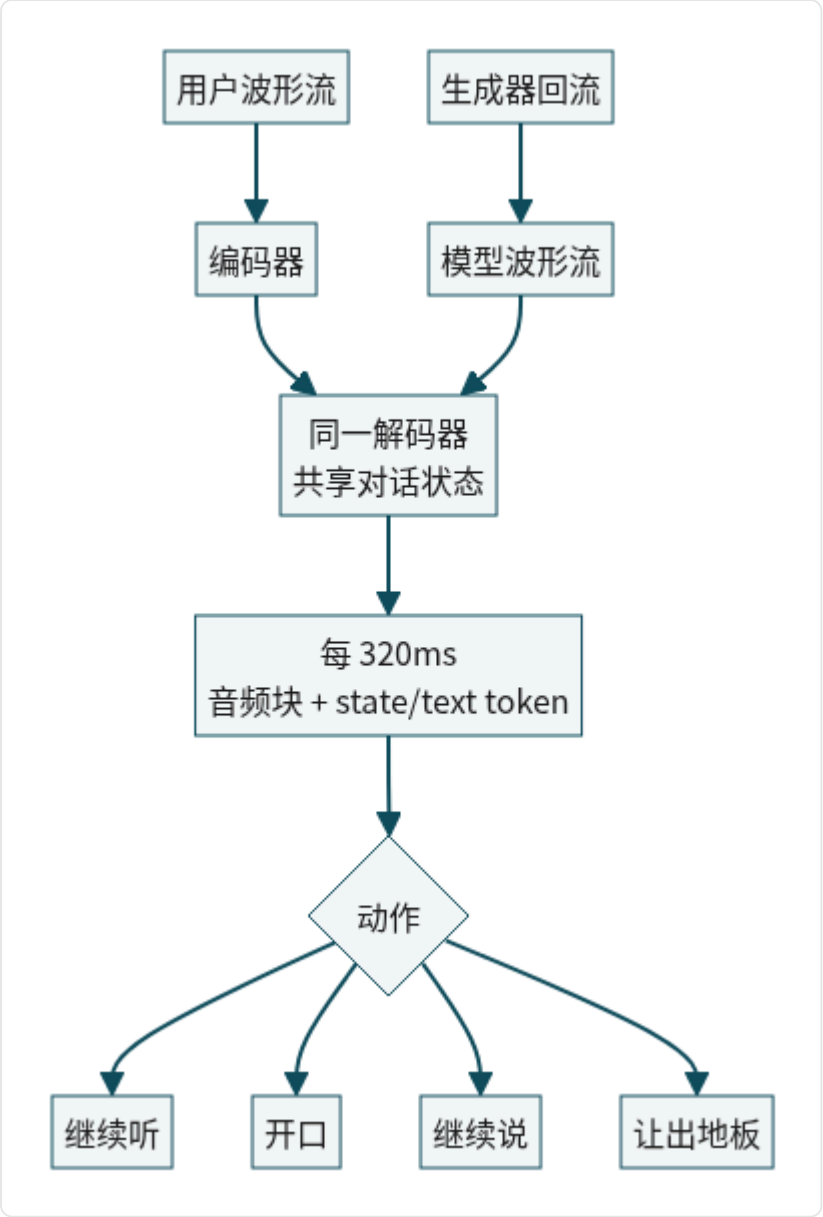
它不是一个更大的自回归序列硬扛一切，而是**在共享对话状态上，把何时听、何时说、想多久、何时行动拆开调度**。

7.4.1 Deep Perception：把听见从转写升级成证据

实时对话不能只做 ASR。情绪、事件、说话人、时间结构、音乐、音频推理，都会改变该不该接和该怎么说。系统甚至在监督阶段分出转写专精分支与实时理解分支：前者冲识别率，后者冲广义音频理解。相对只靠 codec 语义码本顺便听懂，这是把感知做成一等公民——对应四问里的 What to hear，并为后面地板决策提供语义完整度，而不只是能量起伏。

7.4.2 Seamless Duplex：把地板管理写成可监督的时间交错

这是全双工心脏，也是相对 Moshi / FlexDuo 的关键配方差异。



Seamless Duplex 状态决策

音频按约 **320 毫秒**一块，每块后跟一个状态或文本词元。声学证据加语义上下文，引导模型继续听、发起回复、继续说，或让出地板。更精妙的一点：模型用**自己正在说的内容**去解释重叠的用户话——因为用户此刻听到的，正是你嘴上那半句。没有自我播报这一条件，附和与打断极易混淆。

它要显式区分三类、也是四轴里最酸的冲突：

易混对	靠什么判	备注
句中停顿 vs 话轮结束	声学 timing + 语义完整度	不只靠静音门限
附和 vs 真打断	相对正在说的回复判断用户角色	Dual 内继续 / 让路
目标说话 vs 背景人声	结合历史做背景拒识	Idle 思想的内化版

训练上，中期加入流式识别、端点、完整度预测，以及大规模合成全双工；后期用高质量交互打磨接话、附和、打断、拒识。相对 Moshi：Moshi 把地板隐进多流静音/开口分布，细帧分辨率高，但四轴冲突难单独监督；相对 FlexDuo：同属显式态思想，但 FlexDuo 是 L0 外挂控半双工后端，这里是 **L2 内化进同一音频语言模型**，内容生成与地板决策不再割裂成两个系统。

320 毫秒是一种分辨率预算：比 80 毫秒粗，比一秒的世界窗细；极短附和可能吃亏，长程策略更好教。没有免费的时间分辨率。

在全双工基准上，这类地板管理可以冲到很高的综合分（含接话、打断、停顿、附和的均衡画像）。但必须立刻泼冷水：**地板分高 ≠ 指令遵循强 ≠ 工具任务强**。 可达的 Dual 态被架构打开了，实现度仍受数据与路由质量约束——这正是实现裂缝。

用酸测格翻译 Seamless Duplex 的三类区分：停顿 vs 结束 ≈ 犹豫格（需要 Wait 思想）；附和 vs 打断 ≈ Dual 内走继续还是让路；背景拒识 ≈ 第三方格走忽略。它把 FlexDuo 在 L0 用显式态做的事，搬进 L2 的可监督时间线；同时仍主要依赖合成双工与后训练覆盖——若合成分布里附和/第三方不足，分数再高也可能在真实吵杂场景露馅。

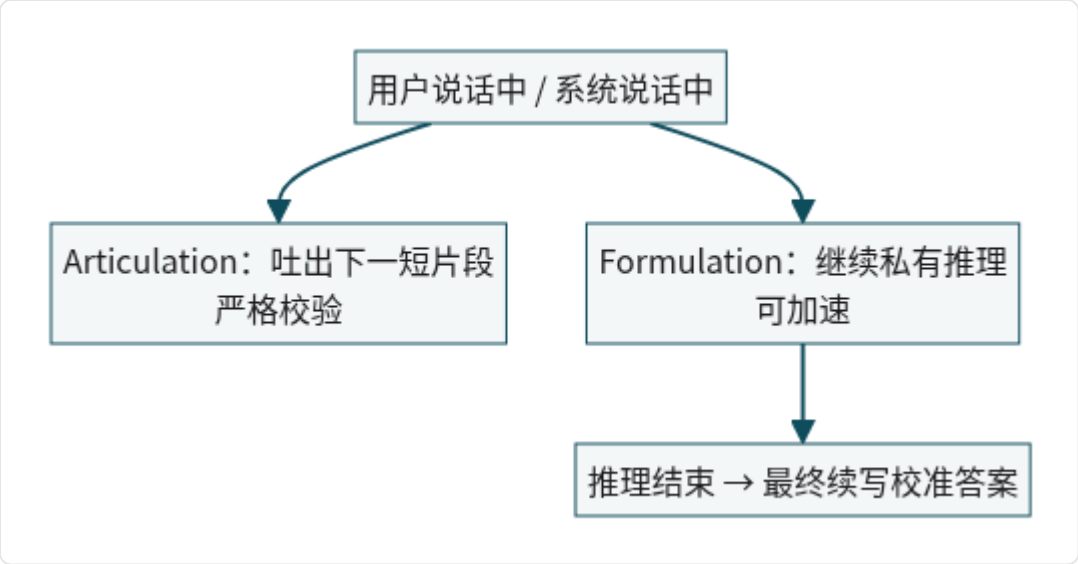
7.4.3 Think-While-Speaking：把延迟 vs 智能拆成两条时间线

这是对第三大矛盾的正面算法回答。灵感来自思维与开口不必同步推进的双过程设计：对**同一个**音频模型做两次并发调用——

脑	输入条件	输出	用户是否听见
Formulation Brain	对话上下文	私有推理轨迹	否
Articulation Brain	已有推理 + 已说出内容	短回复片段	是

调度算法的关键不是两个模型，而是**播放感知的放行**：

时间线	在干什么
Formulation	持续私有推理，用户听不见
Articulation	按已想出部分 + 已说出口内容，吐短片段
播放进度	片段按真实播放放行；想完后最终续写校准



Think-While-Speaking 播放放行

1. Articulation 只产出短片段，条件于此刻已经想出来的部分与嘴里已经答应出去的字；
2. 片段按流式播放进度释放，思考继续；
3. 思考结束后，剩余回答可用完整推理状态；
4. **最终续写**可以补充或修正想一半就开始说的答案；
5. 默认 **Speak-First**（先开口，不等初始推理前缀）；也可 Think-First（先等一小段推理再开口）。

Speak-First 换来体感低延迟，代价是：用户可能先听到基于不完整推理的表述，只能靠后续续写补救——声波进物理世界不可撤回。这与 MiniCPM-o 的 TAIL 是不同轴上的时间对齐：TAIL 对齐的是文本规划与播放/环境；TWS 对齐的是私有推理与开口。

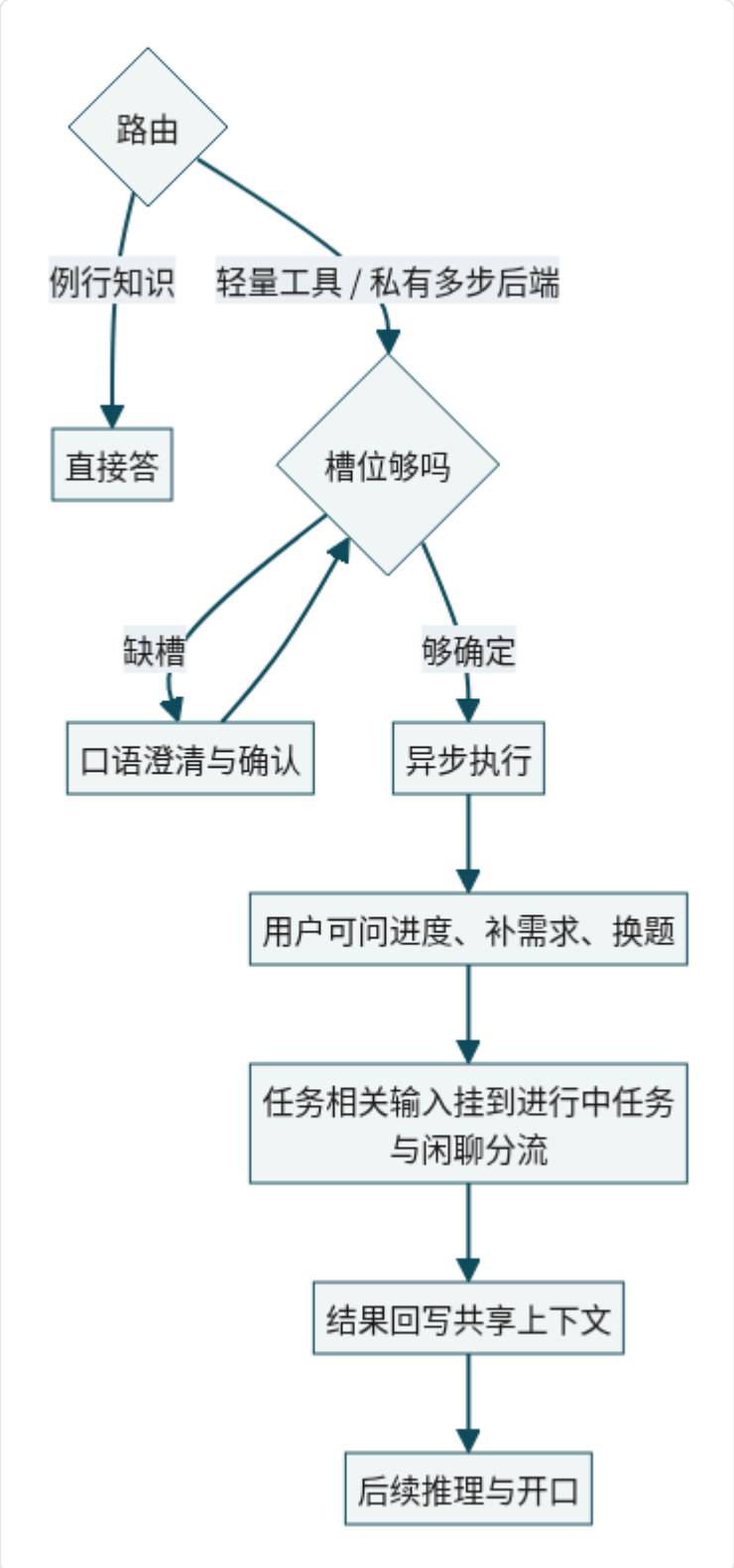
Adaptive Thinking（要不要想） 决定走直接回答还是深思路由。监督构造在助手轮级别：收集上下文、目标答案、原始思维链；用探针模型在空思维条件下重答；盲判有思维是否改变答案质量；再结合任务类型先验，决定哪些轮保留显式推理监督。结果很能说明路由器本身是研究问题：各能力维触发思考的比例大约在一半到八成之间；相对强制不想，推理维受益最大，但自适应路由后推理维分数仍可能低于轮轮都想——说明**省算力的路由会伤到最需要想的题**。语用等维则可能上升。全双工 Agent 的质量，将越来越取决于这类路由器，而不是单点生成分。

MTP 加速只打在私有思维流上：多头草稿减少目标模型逐步解码次数；对真正说出口的语音路径保持严格校验。不对称校验的理由很直接：思维流可冒险加速，声波流不可胡说。

专用推理模式下的对话基准可以很高；叠上 TWS 与自适应后的实时模式，仍可落在专用推理模型同档，而不是塌成低延迟空壳。但实时路径上的指令遵循往往明显弱于纯推理模式——又一次证明：**并行调度解决了在场，不自动保全所有能力维**。

7.4.4 Voice Agent：把行动也放进同一条时间闭环

外部任务常常长于当前话轮。若工具一调用就静音等待，全双工当场死亡。Voice Agent 的算法结构是：



Voice Agent 异步工具路由

TWS 支持想的时候还能说；异步执行支持办事的时候还能聊。两者叠在一起，才是 listen–converse–think–act 的完整时间图像。工具轨迹要用正例教路由与澄清，也要用负例抑制乱调工具与谎称成功。任务成功率评测上，电信等域可以较强，零售与多轮约束跟随仍可能明显落后——说明 Agent 全双工的短板，常常不在能不能打断，而在长程约束是否扛得住。

7.4.5 参数合并：工程整合，不是交互理论

多教师在不同数据配比上分别变强（对话、音频理解、文本推理、混合），再按权重做参数平均合并。它不引入推理期路由模块，成本低，能把多维能力拼回一个权重；但不能幻想平均后每一维都超过最强教师。对 blog 主线而言，它是能力装配的工程手段，真正的交互理论仍在双流状态、TWS 与异步工具上。

7.5 阶段收束

工作	推进点	层级 / 形态
Moshi	取消假轮次，把双工写进生成分布	L2 多流
FlexDuo	把地板外挂成 L0，换可复用与可审计	L0
Step-Audio 2	先把听懂 / 会说 / 会用工具做强	C4；尚非原生双工核
接地推理	保证想不是文本幻觉	C4
StepAudio 3	状态交错内化地板 + 双时间线和解延迟-智能 + 异步行动	L2；F3+F4

收束一下：

> StepAudio 3 不是更强的 Moshi 复刻，而是承认三大矛盾必须分模块调度：感知提供证据，状态交错管地板，边想边说管智商税，异步工具管世界行动——全部钉在同一条连续时间闭环上。

八、S5c：不换双工骨架——横向补人设、音色与跨语文本流（C1 / C2）

8.0 阶段定位

项	内容
账单	固定人设/音色；英语分词器绑定；缺真双通道跨语数据
形态	F3 + C1（人设音色）/ C2（跨脚本）
代表工作	**PersonaPlex；Human-1**
关系	平行分支，不是前后补丁

8.1 本阶段代表工作

工作	推进点
PersonaPlex	Hybrid System Prompt：音色示范 + 角色卡，不改 Moshi 主干
Human-1	换 SentencePiece、重 init 词表参数、26k h 真双通道；交互节奏可迁、语义仍鸿沟

Moshi 骨架能双工之后，产品立刻要两样东西：换角色换音色，以及服务非英语世界。

8.2 PersonaPlex：用时间前缀同时塞进像谁与扮演谁

全双工模型通常固定助手嗓音。延迟约束下，又很难像文本 LLM 那样中途随意改系统提示，也很难像 TTS 那样只靠短音频就克隆，还要两者同时成立。

PersonaPlex 不重做 codec 与多流内核，只在对话开始前拼接混合系统提示：

时间段	助手音频	助手文本	用户侧	作用
Voice Prompt	说话人样本	PAD	占位声波	绑定音色
短暂停顿	—	—	—	分隔
Text Prompt	静音	角色说明	—	绑定人设
短暂停顿	—	—	—	分隔
真正对话	正常生成	Inner Monologue	真实用户	开始交互

系统提示段不反传损失，避免模型把提示本身当成要模仿的对话内容乱学。能力的另一半是数据引擎：大模型写客服与问答剧本，多说话人合成渲染，用静音与负时长静音编排重叠与打断，再叠加真对话与多粒度提示标注。

为什么合理？因为在实时双工里，**条件信息最适合放在时间轴前缀**，而不是塞进每一帧的旁路控制器。它把LLM 式指令跟随和TTS 式音色克隆变成同一多流接口可消费的上下文。局限是合成分布偏差、角色遵循仍可能逊于顶级闭源、工具与安全对齐仍要另做——而且后面会看到，若用风格不对的真人闲聊数据做交互强化，可能冲掉安全人设。

8.3 Human-1：跨脚本时，真正坏掉的是文本流

把双工迁到印地语，表面像微调，实际是问题三角：

1. 英语分词器没有天城文，Inner Monologue 的文本流直接失效；
2. 换分词器使所有词表相关参数作废，不是普通微调，是部分重预训练；
3. 缺少大规模原生双通道自然对话。

菜谱因此非常具体：冻结 codec（并验证跨语重建仍可用），训练新的句子片分词器，重初始化所有词表相关参数，只训练 RQ-Transformer；先在两万六千小时级原生双通道上做高学习率部分预训练，再在精选约千小时上微调并早停。

实验里最值得记住的，不是印地语也能说话，而是：

> **交互节奏可以从真双通道数据里学到，甚至接近或超过部分商用系统；语义内容却仍可能远落后于大闭源助手。**

这句话拆开了两个常被混为一谈的能力：交互智能与语义智能。节奏像人，不等于脑子像人。对研究与产品都是清醒剂：继续只堆伪双流合成，补的是格式；要话轮自然度，需要质量可控的真人立体声；要内容，还得回到更大语言能力、检索与工具。

九、S6：对象从话轮升到世界——环境时间窗上的主动双工（F5）

9.0 阶段定位

项	内容
账单	生成期吸不进新输入（Blocked-I/O）；只会被动应答；端侧要跑得动
形态	F5 世界/多模态双工
代表工作	**MiniCPM-o 4.5**（效率底座 **MiniCPM4** 为隐藏前提）

9.1 本阶段代表工作

工作	角色
MiniCPM4	端侧效率底座（本身非双工论文）
MiniCPM-o 4.5	Omni-Flow 窗更新 + TAIL 播放对齐 + 主动

前面多数系统的交互对象仍是用户话轮。真实助手却活在持续演化的视听环境里。本阶段把问题诊断成两句：

1. **Blocked I/O**：感知与响应交替，生成时吸不进新输入，答的是旧世界；
2. **Passive**：没有显式请求就不行动。

9.2 架构铁律：文本慢通路，语音快通路

端到端约九十亿参数量级：流式视觉编码、流式音频编码、语言模型只产文本与隐状态、轻量语音 token 解码器再接流匹配到波形。铁律是——**大语言模型不要直接吐每秒几十个语音 token**。人语速大约每秒三到四个词片，让大模型按这个速率出文本；韵律决策进入隐状态，小解码器专管声学。否则语言能力与实时性双伤。

多模态系统提示可含文本与参考音频，以支持零样本音色条件。系统同时保留传统轮次模式，兼容旧应用。

9.3 Omni-Flow：把对话改写成窗上的局部更新

灵感是时分复用。连续交互切成固定时长的窗，每个窗吸收新的视觉与音频，并决定是否产出下一段输出。三角色被重定义：

流	角色
环境视觉	持续世界观察
环境音频	声景；用户说话只是世界状态的一部分，不再是特权会话角色
输出流	助手文本与语音，与感知耦合演化，自决定是否开口

每一窗序列化为视觉；音频；输出。无输出时只放监听标记。窗内顺序必须先感知后输出，保证每步说话都吃到最新观察。减小窗长会让感知刷新更快，但过短会导致窗内信息不足，控制与生成一起崩。消融里大约一秒的窗往往优于更短窗；显式边界优于隐式边界；先做听/说二元控制再生成内容，优于把控制缠在文本里——这与外挂状态机控制与内容解耦的结论同向，只是机制内化进同一序列。

窗	视觉 V	音频 A	输出 O
窗1	看	听	说 / 生成
窗2	看	听	说 / 生成
窗3	看	听	listen（选择继续听）
…	持续刷新	持续刷新	先感知，再决定是否开口

主动行为是同一循环的副产品：每窗都可以决定没有人点名，但我要提醒。

9.4 TAIL：嘴必须跟上世界，不能跟着过时的文本跑

即便输入输出同轴演化，仍有第二类时间错位：文本生成速度不等于语音播放速度。若在一秒内生成了要念三秒的文本，嘴会越来越落后于模型当前对世界的理解，于是开始说旧话。固定文本-语音交错比假设时长近似常数，在全双工下仍易过时。

TAIL 按整段交互的累计播放进度，调节本窗该生成多少文本：前面已经落后，本窗就少产文本让嘴追上；监督则按真实发音时间戳把词分配进对应窗。对发音依赖下文的情况，允许有限前瞻，把本窗末尾若干语音 token 推迟到下一窗，而不让文本大幅超前播放。

产品选择很清楚：追求全双工时间对齐，选动态 TAIL；追求纯合成音质基准，固定交错可能更好。它与边想边说不是同一件事——后者并行的是私有推理与开口，前者对齐的是文本规划、播放进度与环境刷新。

9.5 端侧是隐藏主线

没有高效推理与稀疏注意力、量化、推测解码一类效率底座，全模态全双工很难进入可携带设备。交互范式的下一步，往往先被算力卡住。这不是边角料，是产品形态的约束条件。

十、S7：对着实现裂缝刷四轴——似然学不会的，用后训练对齐补回来（C3）

10.0 阶段定位

项	内容
账单	能双工 ≠ 四轴都对；Pause↔Turn 冲突；timing RL 易伤语义
形态	F3 + C3（后训练对齐）
代表工作	**Multi-Faceted Interactivity Alignment（四轴 GRPO）**
与 S5a/S5b	外挂控制 / 训练内化状态 / 后训练 RL 是三角，不是互斥

10.1 本阶段代表工作

工作	角色
Interactivity Alignment	真人双通道切四轴片段 + 轴奖励 + Rllm + GRPO；可叠 Moshi / PersonaPlex

系统已经能并行听说，甚至换角色、换语言、看世界、调工具。但只做词元似然，仍常过静、乱接、缺附和、打断慢；Pause 与 Turn 互相打架。于是出现第三条把交互弄对的路，与外挂控制、训练内化状态并列：

路线	做法	特点
外挂控制	FSM / 个性化 VAD / 语义终点	可插拔、可审计
训练期内化	状态 token + 合成双工	写进同一模型时间线
后训练对齐	按四轴切真人片段 + 轴奖励 + 语义奖励 + 相对策略优化	专刷实现度，不换 L 层

10.2 数据：用真人双通道切出四轴短片段

来源是电话与即兴/自然对话等真人立体声。先用端点检测得到间隙，再按规则切段：句中停顿且对方全程安静，用于 Pause；用户结束后短间隙内对方接话，用于 Turn；长叙述中只有短促反馈，用于 Backchannel；四人轮模式里发生打断再回应，用于 Interruption。每轴控制规模，避免某一轴淹没优化。

刻意少用纯合成对话做这类对齐，是因为交互 timing 对合成偏见极敏感；这与用人设合成教角色可以并存——教的东西不同，数据偏好就不同。

10.3 奖励：看波形，梯度走文本

轴奖励全部在解码后的波形上算：Pause 若出现过长语音则罚；Turn/Interruption 用负延迟；Backchannel 用与真附和时间窗匹配的 F1。对最容易伤内容的延迟轴，再加语义奖励：先识别，再用强文本模型打相关性与自然度，并与延迟奖励分别标准化再相加，避免量纲吞掉另一方。

策略优化采用组内相对优势：同一条件下采样一组完成，比较谁更好。关键实现细节是——**重要性采样与损失主要落在并行文本流上**，因为交互与内容很大程度上由内心文本流决定何时开口；奖励却看声音，从而约束心里想的时机与嘴里出的声一致。

上下文长度从零逐渐加到数十秒，迫使短片段上学会的行为泛化到多轮。消融非常干净：去掉 Pause 数据，乱说爆炸；去掉 Turn 数据，过保守；去掉语义奖励，打断内容分下滑。成功时，Pause 与 Turn 可以同时变好——说明模型开始区分句中停与该接话，而不是简单变得更敢说或更不敢说。

10.4 必须警惕的副作用

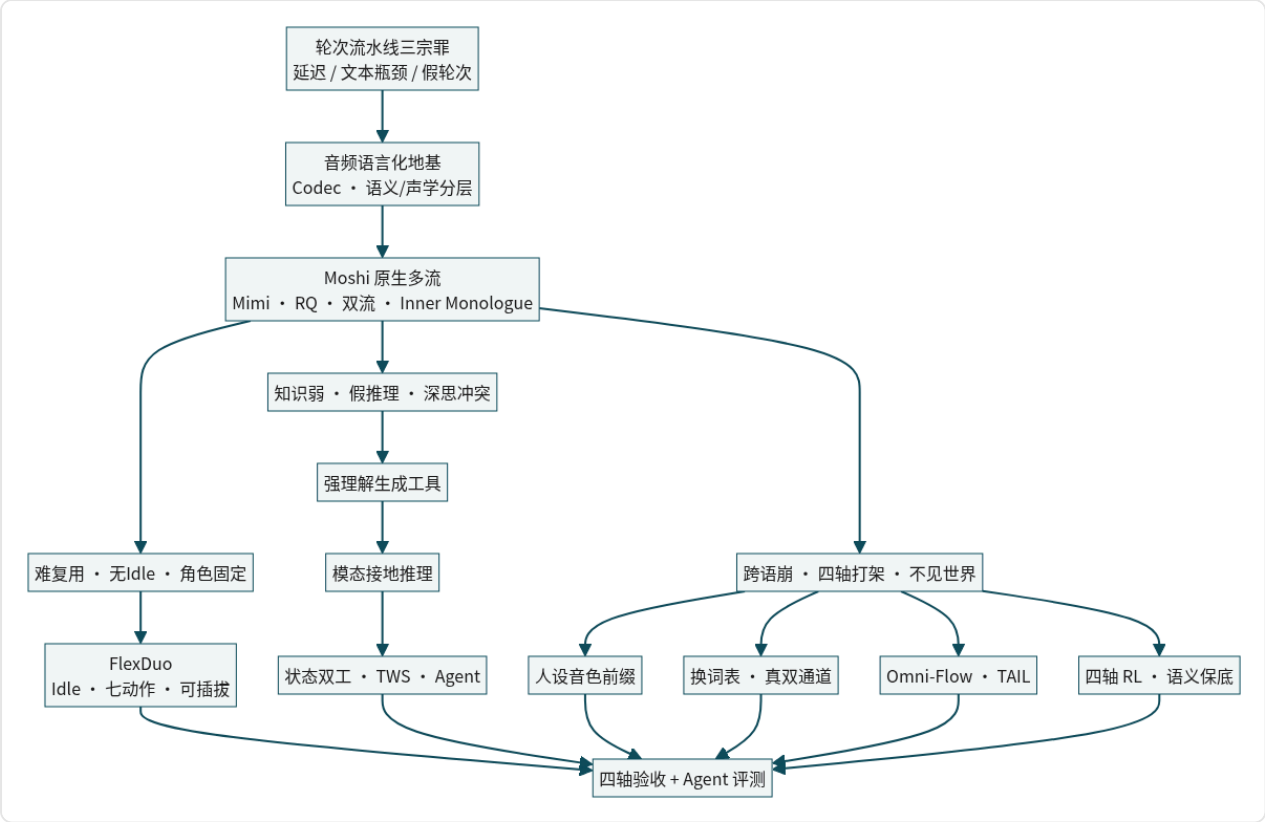
交互数据会改写产品人格。用随意电话闲聊风格去强化带安全角色的模型，可能让模型更善听，却在安全与指令遵循上退步。对齐交互与对齐安全，不是自动同向。未来要把智能、安全与四轴放进同一张奖励表，而不是只优 timing。

这条路也有边界：规则奖励难覆盖笑声、犹豫、文化差异；依赖并行文本流的假设迁移到无内心独白架构时未验证；它不替代 Idle 式背景拒识，也不替代边想边说与工具。

放到实现裂缝框架里：交互 RL 干的是在已有 L2 天花板下，把训练覆盖不足的酸测格用奖励再刷一遍。它提高的是实现度，不改变决策落在哪一层；也很难凭空创造持续并发格——若数据与奖励里没有稳定的 Dual 自环监督，模型不会魔法般学会双方长时间同时说话。

十一、回看总图：阶段如何逼出模块，模块如何逼出下一阶段

发展线若只记模型名，会忘掉因果。下面这张图强调的是问题逼出模块，模块又暴露新问题。



问题逼出模块，模块又暴露新问题

同一条因果链也可以收成阶段表：

阶段	做了什么	随之暴露
问题	轮次流水线：慢、丢副语言、假轮次	—
地基	Codec + 语义/声学分层，音频可语言建模	仍可能 Half-Duplex
Moshi	Helium + Mimi + RQ + 双流取消 turn + Inner Monologue	难复用、无 Idle、角色固定、跨语崩、知识弱、四轴打架、不能深思、看不见世界、不会办事
FlexDuo	Idle + 120ms FSM 外挂	可控可插拔；内容仍弱
能力火箭	latent 理解 + 交错生成 + 工具 → 接地推理 → 状态双工 + 边想边说 + Agent	路由、实时指令、工具长程约束
条件化 / 迁移	混合前缀；换分词器 + 真双通道	节奏可迁，语义仍鸿沟
世界双工	时间窗局部更新 + 播放对齐 + 端侧效率	窗长、主动打扰
交互对齐	四轴奖励 + 语义保底	timing 可对齐，仍非全能

验收贯穿全程：Pause / Turn / Backchannel / Interruption；再向上还有真实不流畅、工具成功、人格与安全。

机制差异也可以收成对照表（不是排名，是你买到的机制）：

系统	双工如何实现	文本在干什么	独特模块
Moshi	80ms 多流自回归	内心独白脚手架	Mimi / RQ / 双流
FlexDuo	外挂三态机控半双工	动作特殊词元	Idle + 七动作
Step-Audio 2	部署偏 VAD	交错 text/audio	工具 / 检索 / 工业理解
StepAudio 3	320ms 状态交错双流	推理流 + 回复流	TWS + Voice Agent
PersonaPlex	继承 Moshi	角色卡进前缀	Hybrid Prompt
Human-1	继承 Moshi	新脚本分词器	真双通道迁移菜谱
MiniCPM-o	秒级视听窗局部更新	LLM 只出慢文本	Omni-Flow + TAIL
交互四轴 RL	改已有双工策略	梯度走文本流	轴奖励 + 语义奖励

十二、逻辑概括：简单，但必须说透

第一，**全双工的本质是时间**。 不是更大的模型，不是更低的首包延迟口号，而是系统在出声时仍更新对世界的感知，并允许重叠事件改写下一步行为。

第二，**音频生成是入口，不是终点**。 没有离散或连续的音频语言化，就没有 spoken LLM；但有了音频生成，仍可能停留在半双工。

第三，**取消假轮次，比更快检测轮次更根本**。 Moshi 用多流联合分布做到这一点；控制器路线用高频状态机近似这一点；两者都能带来体验，优化对象不同。

第四，**控制与内容解耦是反复出现的设计真理**。 FlexDuo 的三态、Omni-Flow 的听/说控制、交互 RL 让文本流管时机，都在说同一件事：把是否开口和开口说什么缠死，四轴就会学崩。

第五，**语言脚手架必须服从实时**。 整段先写再念会破坏双工；按帧内心独白、或慢文本通路加快语音通路，才是实时世界里的合理形态。

第六，**交互不会顺便学会**。 Pause 与 Turn 冲突，Backchannel 与抢答只有一线之隔。专用数据、显式状态或后训练对齐，是必要成本。

第七，**交互像人 ≠ 内容像人**。 真双通道能教会 timing；语义智能仍要靠语言规模、检索、工具与接地推理。

第八，**深思与开口必须并行**。 串行等待是回到半双工的智商税；但并行必须伴随想得对与说出口的校验。

第九，**世界比话轮更大**。 当视觉与声景持续涌入，对话应被改写成时间轴上的局部更新，用户只是环境的一部分。

第十，**终局更像混合实时状态空间智能体**。 模型负责理解与行为，运行时负责确定性、安全与 SLA。纯端到端宗教与纯堆砌控制器，都会在某一层撞墙。

第十一，**读系统用三件套，不读口号**。 决策在 L几、数据盖了哪些酸测格、评测演示了哪些格。缺任一维，讨论都会滑向谁更新谁更双工的空转。

第十二，**半双工的形式病灶是 Dual 内不会消歧**。 一有重叠就无条件让路，必然把附和掐死。全双工的最低形式标准，是 Speak→Dual 之后还能在让路 / 继续 / 忽略之间走对分支。

第十三，**先认形态与功能点，再谈模型**。 F1–F5 与横切增强线说明：同叫全双工，算法对象可以完全不同；用 F1 的打断成功率证明 F4 的边想边说，是范畴错误。

十三、现在还没解决什么，下一步可能往哪走

13.1 仍在的硬问题

语义与交互的鸿沟仍在。节奏可以像人，回答可以仍浅。 **统一目标函数**仍缺。听、说、想、调用工具，多靠分阶段与启发式拼接。 **可中断的局部修订**仍弱。多数系统会停、会让、会重开，少有根据新证据改写已经说出的半句并保持连贯。 **评测仍偏切片**。 四轴必要，但长程任务成功、人格一致、打扰成本、安全，仍未形成同等硬度的公共尺子。 **数据仍贵且偏**。 真立体声自然对话昂贵；合成双工有格式、缺文化与冲突；闲聊分布可能冲安全。 **多说话人嘈杂选听**未统一。Idle、个性化 VAD、背景拒识、双流表示，还没有被一个干净理论收编。 **跨语言**只有菜谱，没有通用迁移理论。 **主动多模态**会打扰。会开口不等于该开口。 **自听与回声**仍是架构加前端的叠加题。 **端侧算力**继续决定哪些范式能离开演示。

13.2 更可能发生的方向

数据与评测优先补洞，而不是只堆 L 层。 持续并发（T4）在公开语料与基准两侧同时空洞；第三方格正在被新语料补，但仍薄。下一步更可能是：合成管道定向灌酸测格、第三方注入、跨库对齐、真并发采集，以及带对话奖励的 RL——与后面交互对齐那条线同向。

L3 仍是架构假说，不是明年产品承诺。 候选方向包括：连续潜变量上的流式自回归（绕开每帧 K 码本）、双流联合预测式表征、世界模型条件对话。它们都在尝试让 Dual 成为共享表征里的自然状态，而不是离散词元上的谈判结果；但生态惯性、流式连续生成成熟度、形式定义，三关未过。

混合系统会成为主流：原生音频模型负责感知与生成，外置或内置控制器负责硬约束与安全，工具与记忆走异步。 交互奖励会从规则 VAD 走向可学习的口语奖励模型，以覆盖更细语用。 调度目标会尝试显式建模轴间冲突，而不是轮流刷指标。 流式解码会研究可修订生成：打断后不是丢弃前

缀，而是局部编辑。 多语种双通道数据会变成基础设施，而不是某一语言的论文附录。 声学前端与模型表示会更多联合训练。 Voice Agent 评测会继续逼近真实不流畅与工具成功。 边想边说会要求可验收的延迟边界与路由质量，而不是只展示加速倍数。 交互对齐会与安全、人设联合优化，避免更善听、更不安全。

附章 A · Moshi 的训练日程与推理循环

只理解模块名字还不够。真正决定系统像不像人的，是训练日程如何把能力一层层叠上去，以及推理循环里每一拍到底交换了什么信息。

A.1 训练是能力装配线，不是一次端到端神话

可以把 Moshi 的训练看成装配线：

步	装配什么
1	文本世界知识（Helium）
2	听觉词表（Mimi，可并行）
3	单流语音语言建模（先会续声音）
4	伪双流格式对齐（学会两路输入长什么样）
5	真双通道动力学（学会重叠与打断）
6	助手人格与鲁棒（合成口语 + 声学增强）

为什么不能一步到位？因为优化目标在打架。一开始就喂真重叠双通道，模型可能连基本可懂语音都学不稳；只做单流，又永远学不会对方突然开口的条件分布。伪双流用说话人分离制造一边说话、一边近乎完美静音的假立体声，专门教格式；真双通道再教两边同时有能量时的行为。指令阶段则主动把用户侧弄脏：增益变化、噪声、回声延迟、混响——因为产品里的全双工，从来不是干净近讲麦。

合成指令数据也有自己的逻辑。书面指令数据充满 URL 与条目腔，不适合口述。更合理的做法是：先把文本模型调成会写口语剧本，再用流式合成把剧本变成双声道，覆盖百科闲聊、口误澄清、安全拒答，并显式要求出现附和与短轮次。合成能补覆盖，也会带来合成腔；所以真双通道与合成在证据上不能等价。

A.2 推理循环里的控制把手

线上每一拍，用户侧 token 来自真实编码，不采样；助手侧采样。离线可以两边都采样，用来统计间隙、重叠等对话动力学。对产品而言，还有一个常被低估的控制点：文本流上的边界符。强迫模型采到可以开始下一个词的信号，等于控麦；让它停留在 PAD，等于继续沉默。全双工系统若完全没有可解释的控制点，上线时会非常可怕——你无法在极端情况下把手刹拉起来。

A.3 算力真相

每一拍都要跑一遍大 Temporal。这意味着全双工原生多流，本质上是用持续算力换持续在场。量化可以缓解，但不会改变问题的形状。这也解释了另一条架构为什么让大模型只以人语速出文本、把语音交给小解码器：他们在买回算力预算，好把参数留给世界知识与多模态。

附章 B · FlexDuo 的数据与动作不平衡：控制学习的隐痛

状态机看起来干净，数据却极脏、极不均衡。真实对话里，继续听可能占一半以上，被打断从说到听可能不到百分之一。稀有转移学不稳，上线时恰恰是这些稀有事件决定体感——用户突然抢话，系统是否干净让路。

标注哲学也很关键：助手接话被要求严格落在用户结束之后；用户打断的时间被保留；听阶段起点人为延后一小段，用来把短附和与真正抢话分开。这不是小技巧，是把四轴冲突写进监督信号。没有这类偏置，分类器会学会有声就听、有静音就说，重新退化成迟钝 VAD。

再强调边界：FlexDuo 让存量半双工模型用起来像全双工，并不自动让它们生成得像全双工听众。助手侧短附和，往往要另外标注与对齐。控制层可以先上，内容层的双工化仍欠账。

方案	决策逻辑	结果
VAD 半双工	有声→听，静音→说	附和、噪声、句中停顿全被误判
FlexDuo	历史语义 + 上一态 + 窗口 → 七动作	上下文更干净，误打断下降；后端内容与附和能力未必跟上

附章 C · Step 系里表示选择为什么改了路线

Moshi 把一切压进离散多码本自回归，统一且美。工业级理解任务却经常发现：输入侧离散化会丢掉细粒度线索。于是出现分裂式表示——**听的时候尽量连续，说的时候再离散**。编码器冻结在多

任务音频预训上，适配器只负责把帧率降到与语言建模匹配的节奏；语言模型输出交错的文本与音频词元；声码路径用流匹配提升发音与音色相似度。

这种分裂不是美学偏好，是分工：连续输入利于感知，离散输出利于与文本同一套采样与解码基础设施，流匹配利于听感。代价是听懂的表征与说出的表征必须靠训练对齐，否则会出现理解对了却说歪、或说得很顺却没听懂副语言。

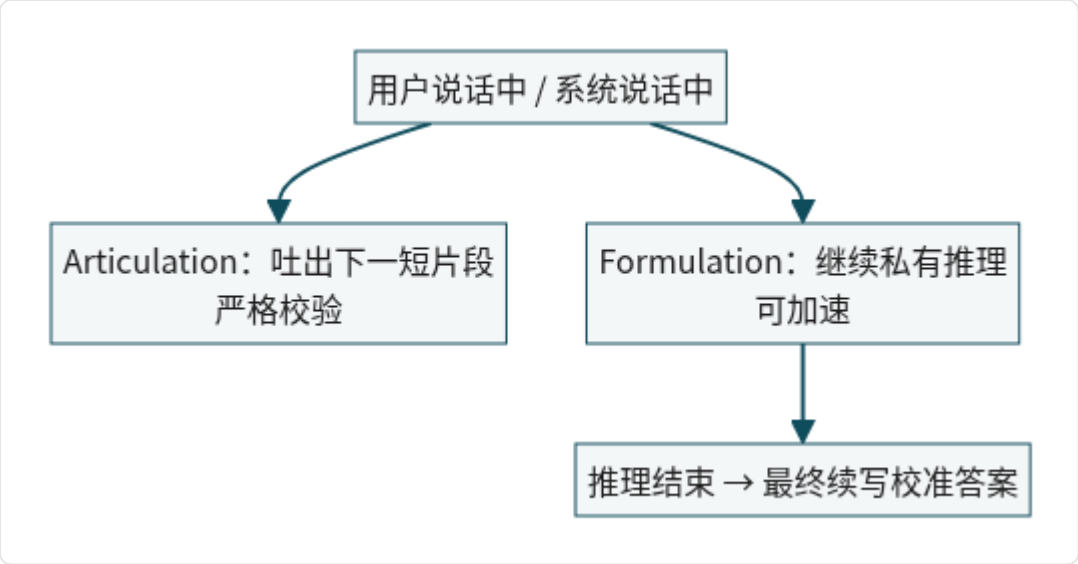
到了实时双工阶段，块大小从 80 毫秒级跳到数百毫秒级状态决策，是另一种权衡：更粗的节拍换来更显式、更好监督的状态空间。极短附和可能吃亏，长程话轮策略更好教。没有免费午餐，只有你把分辨率预算花在哪一层。

时间配方	优点	代价
细帧多码本自回归（如 80ms×深度）	时间分辨率高，重叠细腻	每帧大模型，状态隐式，难监督四轴
块级状态交错（如 320ms + state）	状态可标可训，好打 Pause/Turn	短事件分辨率下降
外挂 120ms FSM	可插拔、可审计	与内容模型割裂

附章 D · Think-While-Speaking 的调度直觉与失败模式

把边想边说想象成两个人共用一张嘴，却有两个笔记本。一个笔记本写私有草稿，用户听不见；另一个笔记本写已经答应说出口的句子。播放器按真实时间把第二本笔记本上的字变成声音；第一本笔记本可以继续往前写。若草稿后来证明开头说得太早或太偏，最终续写可以补救，但无法收回已经进入物理世界的声波——这就是为什么对说出口的路径要严格校验，对私有思维路径可以更激进地加速。

自适应思考则承认：不是所有话轮都值得付推理税。先看不想会不会明显变差，再决定想不想。路由若学歪，会出现荒谬结构：最需要推理的题反而少想，寒暄却在烧算力。全双工 Agent 的质量，将越来越取决于这类路由器，而不是单点生成分数。



TWS 双路径并发

再往算法细节走一步。Adaptive Thinking 的监督信号来自成对对照：同一助手轮，保留原思维链答案，再强迫空思维重答，盲判质量是否变化。这比手写规则数学题要想、寒暄不想更接近数据驱动，但仍受探针模型与盲判噪声影响——所以才会出现推理维最该想，触发率却不够高的结构性失败。

Speak-First 与 Think-First 是延迟-正确性曲面上的两个工作点。产品默认偏 Speak-First，因为口语体感对首包极敏感；高风险域（医疗、法律、资金）更应偏向 Think-First 或提高自适应阈值。最终续写能修正内容，不能收回已播放的错误承诺——若系统刚说可以全额退款再改口，体验损伤往往大于文本聊天里的编辑。

与 L1隐状态旁路出语音相比：TWS 的双脑是**同模并发调用 + 播放门控**，目标是在全双工在场时保留深思；L1 更常服务于流式文本+语音双模态输出，公开谱系里甚至明确有同形状但不宣称全双工的系统。把 Dual-Brain 四个字直接等同于全双工，是常见概念滑坡。

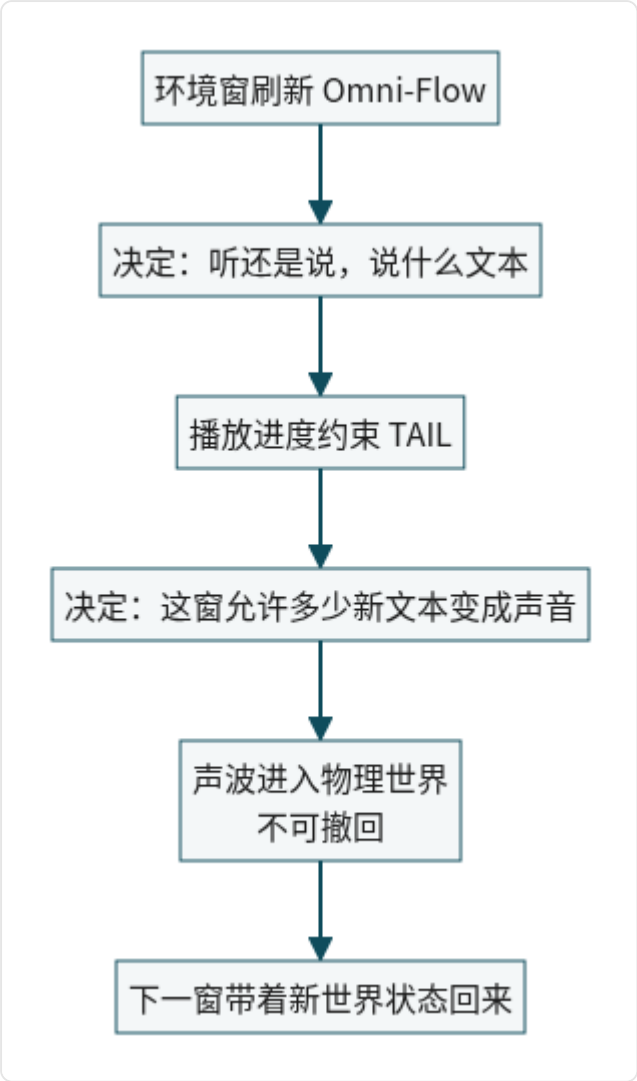
与四轴交互对齐相比：后者改的是何时开口的策略分布；TWS 改的是开口之后智商税怎么交。两者叠加上才完整：先别抢话，再在该说的时候边说边想。

附章 E · Omni-Flow 与双工语音的对象差异

Moshi 族的对象是对话双方。Omni-Flow 的对象是环境。这个差别会改变一切默认值。

在对话对象里，用户请求是特权事件：系统的人生意义是应答。在环境对象里，用户说话只是声景的一部分，视觉变化同样可以触发输出。于是主动不再是外挂规则，而是每窗都在采样：要不要产出非空输出。

窗长选择暴露了另一种冲突：太短，控制与语言任务一起崩，因为模型看不到足够证据；太长，世界已经变了，你还在基于旧窗说话。一秒左右成为经验甜点，不意味着它适合所有任务——体育赛事的瞬间与客服长诉说，对刷新率的需求不同。TAIL 再在语音播放层做一次对齐，防止文本规划跑赢嘴巴。两者叠在一起，才是看得到、跟得上、说得是现在的闭环。



Omni-Flow + TAIL 时间对齐

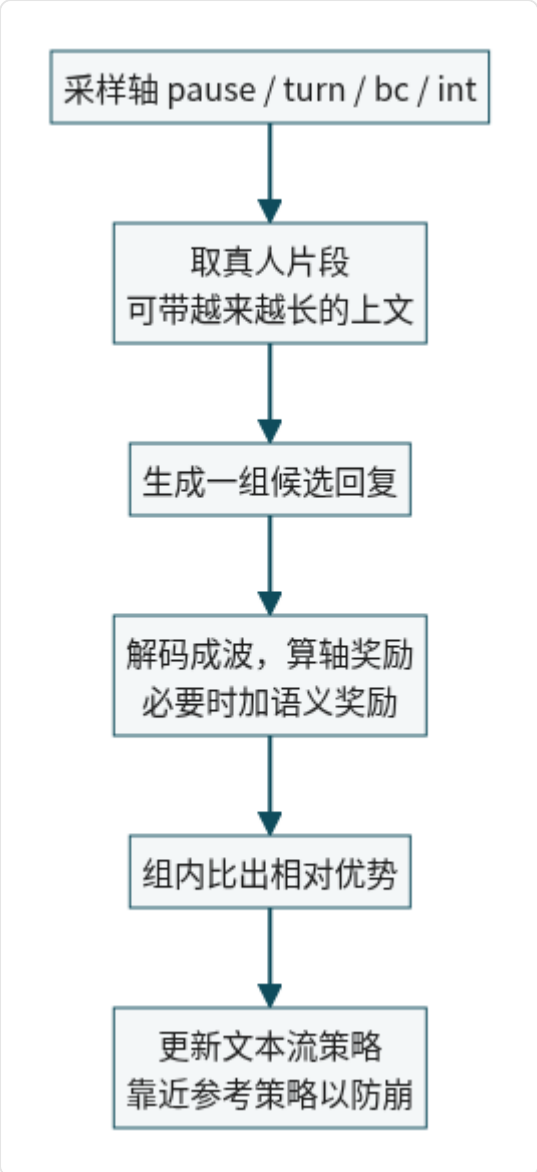
附章 F · 交互强化学习：为什么奖励看波、梯度走文管用

全双工模型若已有并行文本流，文本流往往既写内容，也写时机。波形是最终体验，文本是策略的可微把手。于是出现一种看起来别扭、其实非常合理的分工：用波形定义对错，用文本流计算策略梯度。

四轴联合采样很重要。若今天只训 Pause，模型会学会少说话；明天只训 Turn，又学会抢着说。必须在同一训练过程里随机抽轴，让冲突在梯度里见面。语义奖励不是锦上添花：延迟奖励天然诱

导模型为了快而说废话或说错话。把语义分与延迟分分开标准化再相加，是在防止一个标量奖励里藏进不可见的权重霸权。

上下文课程同样关键。始终在无历史的短段上对齐，模型会成为单次刺激-反应机器；始终在长历史上对齐，又可能一开始就不稳。让最大历史从零长到数十秒，是在教泛化，不是在凑式子。



交互 RL 优化循环

失败模式要写进方法论：交互数据分布会改写安全行为；规则奖励会卡住更细的语用；没有第五轴 Idle 时，背景拒识不会天上掉下来。

附章 G · 一张问题→模块→新问题的细表

暴露的问题	关键模块级回答	随之而来的新问题
波形太长无法建模	Codec 离散化 / 连续编码降帧	语义与音质冲突、跨语损失
纯音频知识弱	文本骨干初始化 + 混训	分词器绑定语言、知识规模上限
多码本展平不可实时	Temporal + Depth 层次	每帧大模型算力、delay 与质量权衡
假轮次无法表达重叠	用户 // 助手多流	无 Idle、伪双流偏差、固定身份
整段文本再念不实时	按帧 Inner Monologue	PAD 为主、对齐噪声、非深度推理
一体化难复用	外挂 FSM + Idle	内容弱、附和弱、双模型延迟
理解与工具不足	latent 输入 + 交错输出 + RAG	仍非原生双工
长思维变差	模态接地推理蒸馏	尚未解决实时开口
深思与低延迟冲突	双脑并行 + 自适应路由	路由次优、说出口不可撤回
固定音色角色	混合系统前缀	合成偏差、安全被后续对齐误伤
跨脚本崩	词表重 init + 真双通道	语义鸿沟暴露得更清楚
生成时世界已变	时间窗局部更新	窗长权衡、主动打扰
文本跑赢播放	按播放进度动态交错	依赖时间戳监督
似然不优四轴	四轴 RL + 语义保底	规则奖励天花板、轴外行为

这张表才是发展线的骨架肌理。模型名字只是表上的行号。

附章 H · 用一段对话，把机制差异说成人话

设想用户说：我昨天……（停两秒）……反正最后没做成。

- 只会 VAD 的半双工：两秒静音触发抢答，打断用户的反正。
- Pause 没对齐好的多流模型：同样可能抢，因为它在数据里见过太多短静音后接话。
- Idle/Listen 状态机：更可能继续听，把两秒当句中停顿；若历史显示用户常犹豫，更不应抢。
- 四轴 RL 后的模型：Pause 奖励惩罚在用户停顿里说话超过阈值，Turn 奖励又要求真结束后快接——它必须在同类样本上学会区分。

再设想用户长篇抱怨，中间需要"嗯"。

- 保守系统：全程沉默，最终答案正确，但体验像审讯。
- 过激多流：把"嗯"说成一句完整安慰，变成抢话。
- 合格 Backchannel：短、低频、落在合适间隙，不改话题。

再设想系统正在读一条很长的方案，用户突然说等等，预算只有一半。

- 半双工播报：很难真停，或停了却听不清。
- 双流/状态双工：用户流并行进入，状态切到听，停止或收束自己的输出。
- 有边想边说与工具的系统：先口头确认约束，再异步改方案，不必让用户听着沉默等到工具返回。
- 仍普遍缺的能力：把已经说出的前半句方案无缝改成那我们改成半预算版本，而不是生硬重开。

三段场景分别压测 Pause、Backchannel、Interruption+Agent。任何只在实验室朗读稿上刷分的系统，都会在这里露馅。

附章 I · 终局图像：Realtime State-Space Agent

传统级联可以写成：用户音频到文本，文本到回复文本，再到语音。信息瓶颈发生在第一道门。全双工更像状态方程：当前状态由上一状态、当前用户音频、自己刚说过的话、环境与工具结果共同更新；再从状态发出声音与动作，甚至改变环境。

符号	含义
(S_t = f(S_{t-1}, 用户音频_t, 自己音频_{t-1}, 环境_t, 工具结果_t))	状态更新
(说话_t = g(S_t))	从状态发出声音
(动作_t = h(S_t))	继续想、调用工具、保持沉默等

这不是某一篇论文的定理，而是帮助工程师选模块的压缩图像：你做的每一个模块，要么在改进状态表示，要么在改进从状态到听/说/想/行的策略，要么在改进状态更新的时间分辨率与可靠性。

于是未来方向也可以说得不空：更好的状态（多模态、可修订、可记忆），更好的策略（四轴+安全+任务成功的统一目标），更好的时间（可中断局部改写，而不是停/续二值），更好的评测（从切片行为到长程相遇）。

附章 J · PersonaPlex 与 Human-1：同一骨架上的两种条件化

条件化听起来像提示工程，在全双工里却是结构性难题。文本 LLM 可以在上下文任意位置塞系统提示；TTS 可以在开头塞参考音频。全双工模型每一拍都在同时读用户、写自己，提示必须变成**时间轴上的合法前缀**，并且不能破坏双流对齐。

PersonaPlex 的混合前缀，本质是两次伪造对话开头：

第一次伪造教音色。助手音频通道放真实说话人样本，文本通道放 PAD，意思是这段时间有声，但没有要念的字——模型把声纹与通道绑定。用户通道放占位信号，避免空通道造成分布外。

第二次伪造教角色。助手文本通道写入角色卡，音频通道静音，意思是这段时间有字，但还没出声——模型把指令绑定到后续生成的内心独白与行为。中间的短暂停顿与分隔符，是在告诉模型：提示结束，生活开始。

损失屏蔽提示段，是为了防止模型学会复读角色卡或把提示当对话内容。这是细节，却决定可控性是接口还是泄漏。

Human-1 的条件化对象不是角色，而是语言本身。Inner Monologue 依赖子词表；子词表一换，所有输入输出嵌入、输出头都要重来。冻结 codec 是在赌：声学词表跨语可迁移；重训 RQ 是在承认：语言与交互动力学必须在新符号系统里重学。两阶段学习率与早停，是在防全量数据教坏精选能力——大规模先找回双工格式与节奏，小规模再对齐质量。

两者对比能看出条件化的层次：

工作	改什么	不改什么	服务目标
PersonaPlex	前缀条件（角色/音色）	符号系统	像谁
Human-1	符号系统（新分词器）	声学词表 / codec	语言/脚本
交互 RL	策略分布（timing）	符号系统	何时开口

三种改法深度不同，成本不同，伤到的能力面也不同。做人设时用前缀；换语言时动词表；修抢话时动奖励。乱用量级，就会用牛刀杀鸡或用针修飞机。

附章 K · 评测如何反向塑造架构

没有四轴评测时，架构讨论容易停在口号：谁延迟低、谁端到端、谁参数大。有了四轴，你才会发现快接和善听可以分裂在不同模型上。

接管率只是尺子的刻度。刻度方向随轴翻转，这逼着研究者不能再用单一标量排行榜偷懒。一个在打断上延迟极低的模型，可能在停顿上是灾难；一个在停顿上很忍的模型，可能几乎不会附和。于

是架构开始分化出显式状态、Idle、背景拒识、四轴 RL——不是为了发论文好玩，是因为评测把冲突测出来了。

当评测再升级到真实不流畅与工具调用，系统又会被逼出边说边调工具、边听边改计划的模块。评测不是事后贴金，是发展线的发动机之一。

还要注意两套常被叫混的尺子：一套以行为场景与多轮考官为主，一套以可配置生成管道为主——名字相近，意图不同。v1.5 把重叠拆成打断/附和/对他人说/背景声，才让附和被误杀这类失败可定位；并观察到系统常分裂成两种策略气质——**响应优先**（快，但易被附和骗停）与**占住地板**（稳，但对真打断慢）。这与 Pause/Turn 互相打架是同一深层结构。另：静态录音集测不到人会根据系统刚才的行为改下一句的递归适应；缺因果探针时，块交替系统也可能靠日程表刷出好看的停延时，却未必真在听。

评测升级	逼出的模块方向
只会内容分	大家都做半双工问答
加入四轴行为分	双流 / FSM / 状态 token / 交互 RL
加入工具 + 真实不流畅	Voice Agent、异步工具、鲁棒端点
未来：长程人格 / 打扰成本 / 可修订生成	新模块几乎可预期

附章 L · 三个设计原则的反例证明

原则一：控制与内容宜解耦。 反例是把听/说缠在同一文本生成里硬学，或只靠似然希望模型自己懂规矩。结果通常是四轴此消彼长。正例是三态机、听说控制词、文本流管时机而波形管奖励。

原则二：时间对齐是第一公民。 反例是文本先跑很远再合成、或生成期阻断感知。结果是说旧话、答旧世界。正例是声学 delay 的谨慎选择、播放感知调度、窗内先感知后输出、按播放进度限制新文本。

原则三：数据决定动力学，目标函数决定冲突如何和解。 反例是只用伪双流或只用合成闲聊，却宣称具备真实打断与安全客服能力。正例是真双通道教节奏、合成教覆盖、四轴联合奖励教冲突和解，并且盯住分布对安全的副作用。

附章 M · 模块级最小可行全双工与完整 Agent清单

若你要做一个最小可行全双工演示，模块可以少，但不能缺类：

最小集	模块
1	流式音频表示（codec 或连续编码）
2	并行听通道（双流或等价的持续预填充）
3	实时生成（低 delay）
4	至少一个话轮策略（隐式静音 / 状态 / 外挂控制）
5	四轴里至少 Pause + Interruption 的验收

完整口语 Agent 在最小集之上还要：

增量	模块
+	语言脚手架或慢文本通路（保内容）
+	感知深度（副语言/事件，而不只转写）
+	显式或可对齐的四轴策略
+	背景拒识 / Idle
+	边想边说调度
+	异步工具与记忆
+	人设与音色条件
+	安全与交互联合对齐
+	声学前端（回声与选听）
+	端侧/成本约束下的效率栈

这份清单的用途是防止用一个亮点模块冒充全系统。Moshi 很强地覆盖了最小集的中心；FlexDuo 强化策略与拒识；Step 系补感知、思考与工具；PersonaPlex/Human-1 补条件化；MiniCPM-o 补世界与对齐播放；交互 RL 补四轴冲突。拼图需要拼，不是互相取消。

附章 N · 回流：发展线的不可跳级

常有团队想跳级：直接做多模态主动全双工 Agent。跳级的代价通常是：话轮还不稳就主动打扰，工具还不会异步就深度推理，角色还不会跟就上线客服。发展线里的顺序有因果：

阶	产品承诺
1	先能实时出声
2	再能边听边出声
3	再能在四轴上不像冒犯
4	再能在深思时仍保持在场
5	再能办事
6	再能看见世界并主动
7	再能在多语言多角色上迁移

不是说研究不能并行，而是说**产品承诺不能跳级**。并行研究出模块，串行组装出体验。这又回到开头那张二维表：你可以在架构轴上跳得很远，同时在交互轴上仍停在半双工；也可以交互轴很炫，内容仍空洞。

附章 O · 机制语言总复习

若只用一张表收束主线，大概是这样：

目标	被迫长出的能力
让演化可计算	声音语言化（codec / token LM）
让演化像对话	并行听与生成，取消假轮次
让演化可控	状态、前缀条件、交互对齐
让演化聪明且仍在场	接地推理、边想边说
让演化进入世界	多模态窗更新、异步工具
让演化可验收	四轴 + 任务成功，而不是只用内容分

每一行都对应可指认的模块；每个模块，又在解决更浅层模块制造或暴露的问题。这不是故事线包装，是约束条件下的工程必然。

十四、设计纪律

1. 先画时间轴，再选网络。帧长、窗长、延迟、状态节拍，决定体验上限。
2. 先分清你在优化 E2E 还是 Duplex，避免用端到端分数证明全双工。
3. 语义与声学分层，或文本慢通路与语音快通路分工，不要让一个头扛所有。
4. 双流解决序列自条件，不替代回声消除。
5. 能解耦控制与内容就解耦；四轴冲突时尤其如此。
6. 真双通道用于教 timing，合成用于教角色与覆盖长尾，不要混用它们的证据强度。
7. 有推理就做模态接地；无接地的长思维会损害音频任务。
8. 要深思就并行调度，并校验说出口的话。
9. 上线前用 Pause/Turn/Backchannel/Interruption 四轴验收，并看它们是否互相牺牲。
10. 把安全与人设和维护交互的数据放在同一张桌子上谈，否则强化一只手，打掉另一只手。
11. 报告系统时写三件套：决策在 L几、训练盖了哪些酸测格、评测演示了哪些格——拒绝只有一个双工总分。
12. 验收全双工最低形式标准：Speak→Dual 之后，能否区分让路 / 继续（附和）/ 忽略（第三方）；只会无条件让路，就还是半双工策略。
13. 选型顺序：场景功能点 → 酸测格/转移 → F 形态 → 具体模型；禁止从模型名倒推需求。

附章 Q · 各阶段论文的诊断对照

把各阶段代表工作放进同一张诊断表，不是为了排名，是为了防止名词更新冒充问题推进。

系统	决策落点	Dual 构造	酸测上常见强弱画像	实现裂缝主要来自
Moshi	L2 并行流	强（一直潜在）	接话/打断强；Pause/附和易过激	Fisher 规模与合成指令分布
FlexDuo	L0 外挂	弱	Idle/噪声过滤强；附和生成弱	半双工后端 + Dual 达不到
FireRedChat	L0+pVAD	弱–中	第三方格相对干净；首包链路仍长	L0 延迟地板
Step-Audio 2	非 FD 主张	—	理解/工具强	部署 VAD，地板未 L2 化
StepAudio 3	L2 状态交错	强	四轴地板可冲高分；指令/零售工具分岔	合成双工覆盖 + 路由
PersonaPlex	L2+前缀	同 Moshi	人设/音色↑；交互遗传 Moshi 优劣	合成剧本分布
Human-1	L2 迁移	同 Moshi	节奏可迁；语义鸿沟尖锐	Type C 有了，智能仍弱
交互四轴 RL	改 L2 策略	—	专刷 Pause/Turn/BC/Intr 实现度	不创造新 L 层；难变出持续并发
MiniCPM-o	L2 式窗序列	视听版	世界刷新与主动；短事件分辨率粗	窗长与播放对齐权衡

三条更深的思考：

0. **同全双工标签可以完全不是同一形态**。 F2 外挂与 F3 多流、F4 边想边说、F5 世界窗，服务的功能点与损失函数不同；前面的 F 形态表应与本表对照看。
1. **同 L 层可以完全不像**。 Moshi 与 StepAudio 3 都是 L2，一个把 Dual 隐进多码本联合分布，一个把地板写成 320ms 状态监督；优化的可审计性与时间分辨率交易不同。
2. **数据 Type C 决定话轮，语言规模决定内容**。 Human-1 把这一点烧进结果：节奏像人、内容仍可能远落后。
3. **L3 空行提醒我们：现在所有原生双工仍在用离散或块级符号谈判时间**。 真正共享连续潜空间里的 Dual，还是研究假说；产品期应把预算优先打在酸测格覆盖与实现裂缝收敛上。

附章 P · 事实边界与易错点

核对原则：以公开技术报告与可核对数字为准；产品叙事降级；不把结构相似写成能力等同。

已核对为稳妥的表述

- 全双工 ≠ 端到端、Streaming ≠ Full-Duplex：成立，且与交互评测实践一致。
- Moshi：多流、Mimi 约 12.5Hz/8 码本、Inner Monologue、理论延迟约 160ms、实测约 200ms量级：与公开技术报告一致。
- FlexDuo：Idle + 外挂控制、相对一体化基线降低误打断并提升接话：方向与论文一致；具体百分比以原文表为准，文中用约数叙述。
- Step-Audio 2：连续输入 + 离散音频输出交错、部署常见 VAD：定位为能力底座而非 Moshi 级多流双工核——这一分类是刻意的，避免夸大。
- StepAudio 3：320ms 状态交错、TWS 双脑同模并发、Adaptive Thinking、MTP 加速私有思维、Voice Agent 异步工具；推理模式 StepAudioChat 宏约 73.0、实时模式约 70.4；全双工综合分约 98.9；τ-Voice 宏约 56.0%：与技术报告一致。
- 架构可达 ≠ 行为实现：双工分高而零售工具/多轮约束/实时指令遵循仍可能偏弱——报告结论段明确承认，文中已保留冷水。

已纠正或需读者警惕的易错点

1. **Dual-Brain / R1.1 流式形态 ≠ 已宣称的原生全双工**。 公开谱系里存在想/说拆分、L1 隐状态形状、但不以全双工地板为主张的系统；更稳妥的做法，是把模态接地推理双脑边想边说 Seamless Duplex 全双工分成三层火箭，避免混名。
2. **双流 ≠ 回声消除**。 文中已强调；部署仍需声学前端。
3. **AA Full-Duplex Bench 高分 ≠ 全面产品胜利**。 文中已拆开地板、指令、工具三张成绩单。
4. **合成双工数据教得了状态，不等于真双通道教出的听者文化**。 Human-1 路线与 Step 合成双工是互补证据，不是互相替代。
5. **L0/L2 不是进化等级**。 L0 外挂仍在工业上活跃；L2 只是决策落点不同。
6. **L0–L3 / 酸测 / 五态等框架用于对照，但不伪造系统未声称的能力**。 L0–L3、酸测六格、五态十一转移、实现裂缝、Type C 数据瓶颈、持续并发空洞、FDB 与 FD-Bench 区分：均对齐公开综述表述；对具体系统的 √/△ 覆盖，正文用常见画像/构造可达叙述，不把综述审计表每一格写成该系统论文原文断言。
7. **FlexDuo Idle 与综述 Idle 功能位置不完全等同**（更偏说完后缓冲），文中已区分，避免混名。
8. **知乎专栏若与综述同构，以上吸收有效；若另有独有实验数字，需原文核对后再写入，不以口传补写**。
9. **F1–F5 是形态分层**，与 L0–L3（决策落点）正交：例如 F2≈常落在 L0，F3/F4≈常落在 L2，但 F4 的想还可呈 L1 形状；勿把 F 序号与 L 序号一一等同。
10. **Step-Audio 2 / 接地推理标为横切 C4 而非 F3**：与仓库分类一致——它们是能力/推理底座，部署仍可偏 VAD，不应写成 Moshi 级原生双工核。
11. **FireRedChat 写入 F2 对照表**：主系列有专章；若读者论文集未收录，仍属系统级全双工典型，不虚构其 FDB Table III 分数。

仍标为开放、避免写死的点

- 专家路由与双工延迟的精确工程剖分、状态词元完整词表：公开文本粒度不足处，不编造。
- 商业闭源系统内部是否 L3：无公开证据则不宣称。

全双工语音交互的故事，不是一条越来越新的模型发布时间线。它是一次又一次被真实对话打脸后，被迫把时间写进模型结构的故事。谁把时间处理对了，谁才配谈论下一句：机器能不能像人一样，在重叠与打断里，仍然听得见、想得清、说得稳，并且知道何时不该说话。

整理自 full-duplex 学习笔记发展线稿。中心主张：全双工 = 连续时间上的状态演化；读系统时看三件套——决策在 L 几、数据盖了哪些酸测格、评测演示了哪些格。